Advanced Computer Graphics (ACG)

Theory Session 2 - Basic Ray Tracing

2026-2027

Pompeu Fabra University

Ricardo Marques (ricardo.marques@upf.edu)

Calendar

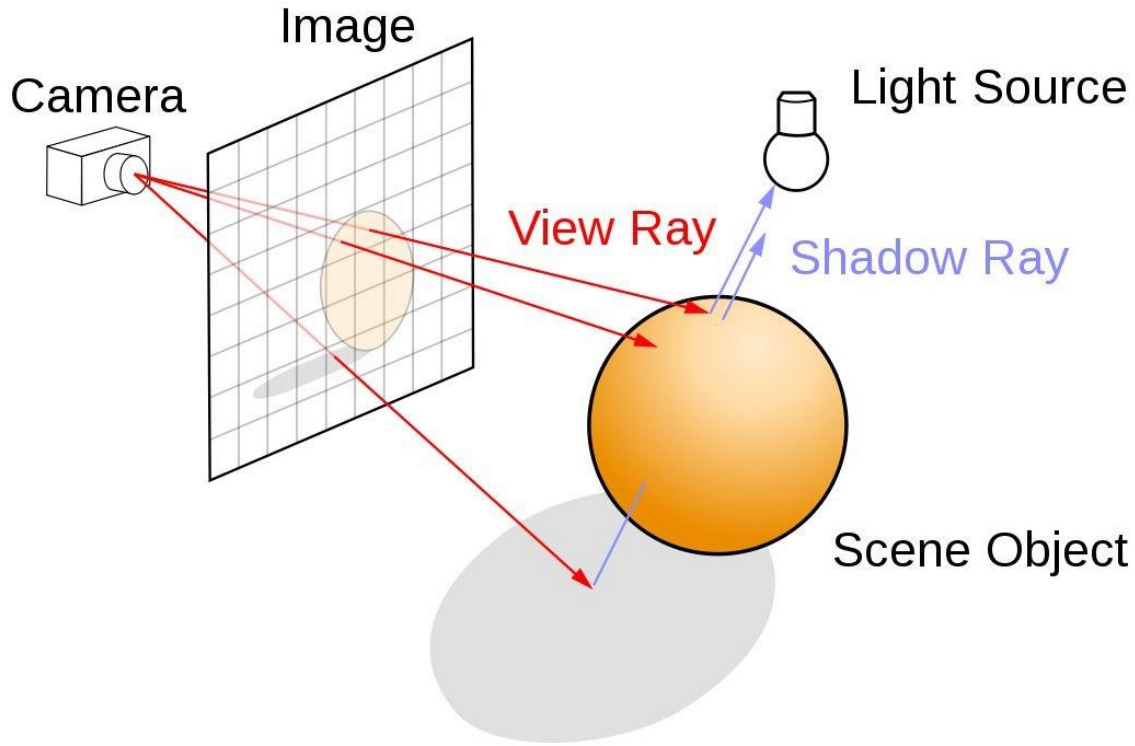
Week no.	Date	Theory	Practice	Seminar	Deliveries
Week 1	28 Sep 02 Oct	**T1**: Introduction	**P1**: Introduction to A1.1	-	
Week 2	05 Oct 09 Oct	**T2**: Basic Ray Tracing		**S1**: Work on A1	
Week 3	12 Oct 16 Oct	**T3**: Ray Casting, Recursive Ray Tracing	**P2**: Introduction to A1.2		
Week 4	19 Oct 23 Oct	**T4**: Rendering Equation, Monte Carlo Integration and Direct Illumination **T5**: Indirect Illumination and Path Tracing		**S2**: Work on A1	**D.A1**
Week 5	26 Oct 30 Oct	**T6**: Volume Rendering: Absorption-Only Model	**P3**: Introduction to A2.1		
Week 6	02 Nov 06 Nov	**T7**: Volume Rendering: Emission, Absorption and Scattering Model **T8:** Neural Radiance Fields and Topics' Presentation		**S3**: Work on A2	
Week 7	09 Nov 13 Nov	**T9**: **Midterm**	**P4**: Introduction to A2.2		
Week 8	16 Nov 20 Nov		**P5**: Work on A2	**S4**: Work on A2	**D.A2**
Week 9	23 Nov 27 Nov	**T10**: **1st Presentation**	**P6**: Work on FP	-	
Week 10	30 Nov 04 Dec	**T11**: **2nd Presentation** **(First Session)** **T12**: **2nd Presentation** **(Second Session)**		**S5**: Work on FP	**D.FP**

Outline

- Part I - Overview of Ray Tracing Algorithms
  - Ray Tracing: Intuition
  - Ray Casting (Apple, 1964)
  - Recursive Ray Tracing (T. Whitted, 1980)
  - Distribution Ray Tracing (Cook, 1984)
  - Path Tracing (Kajiya, 1986)
- Part II – Ray-Object Intersection
  - Ray-plane intersection
  - Ray-triangle intersection

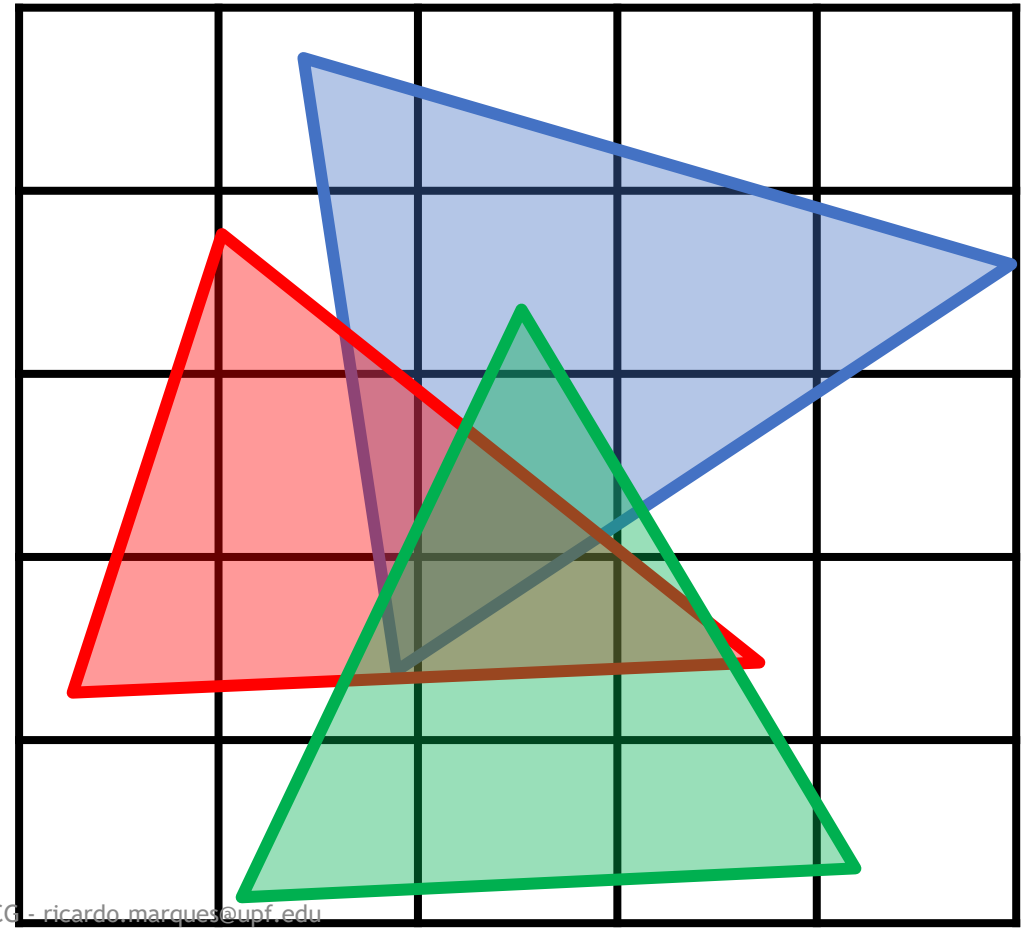
Part I

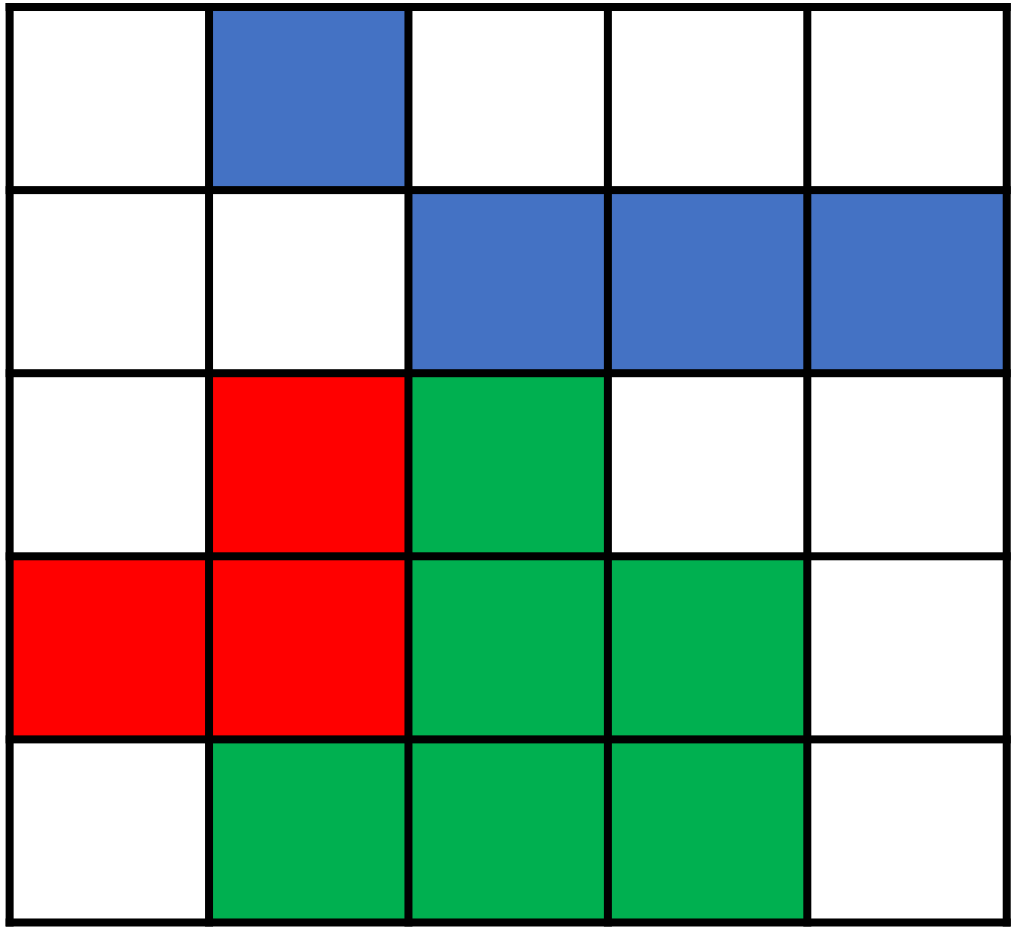
Overview of Ray Tracing Algorithms



Ray Tracing: Intuition

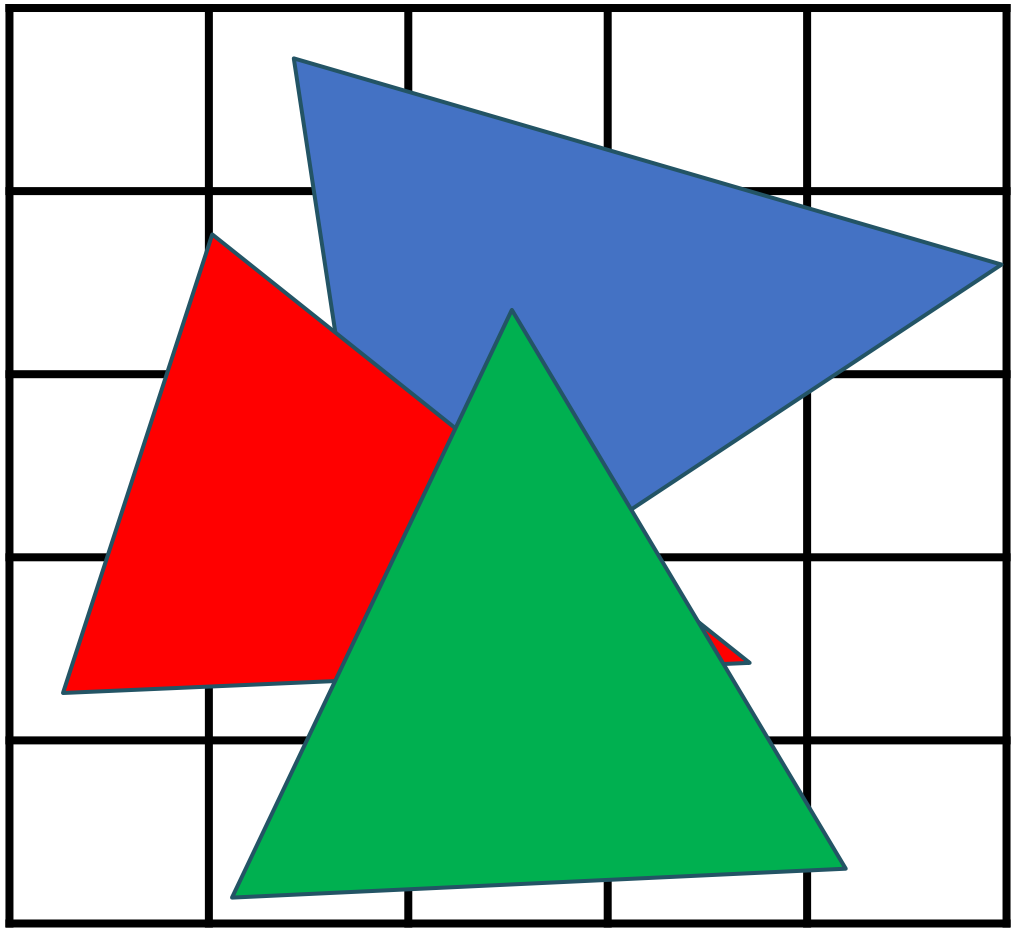
Problem Set-Up



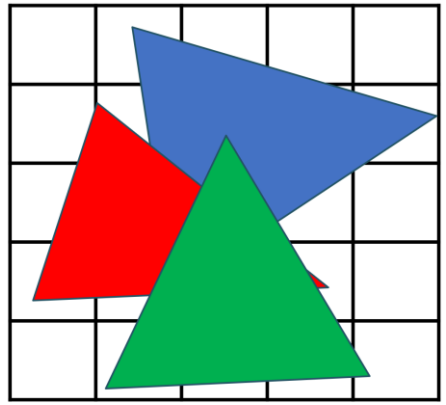


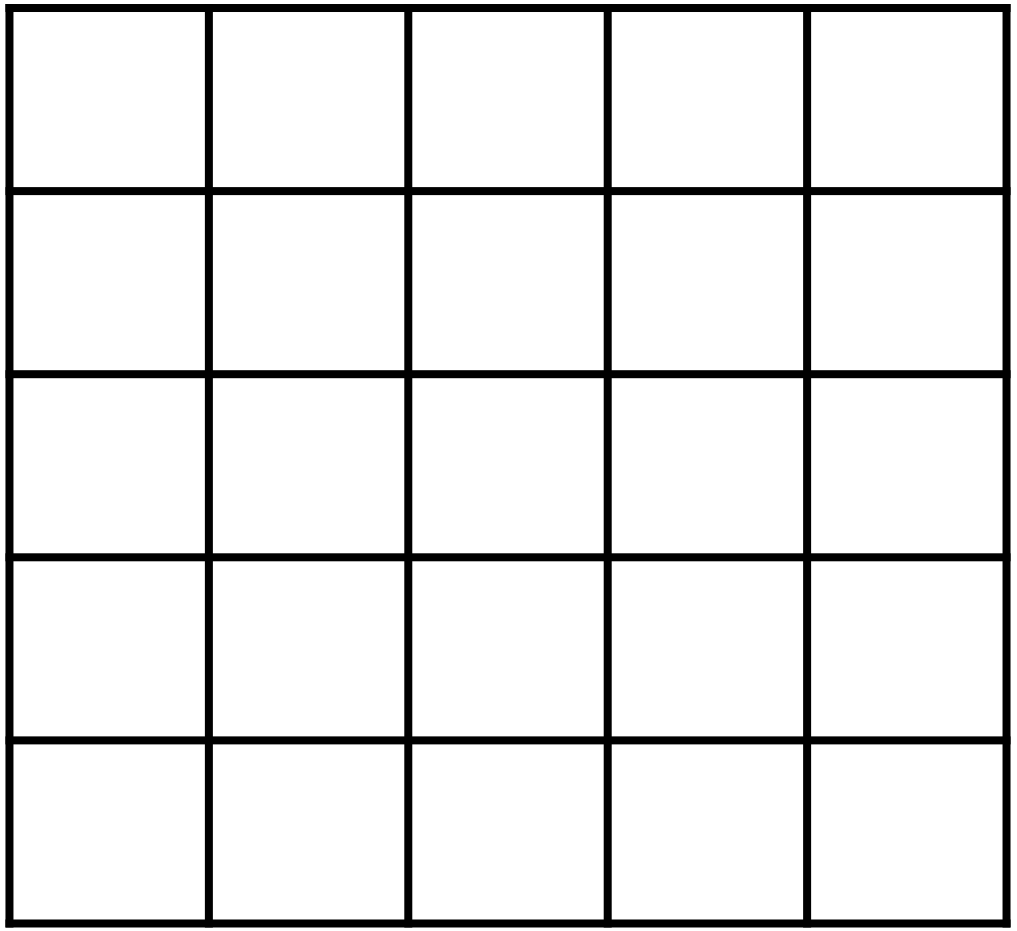
ACG - ricardo.marques@upf.edu

Rasterization

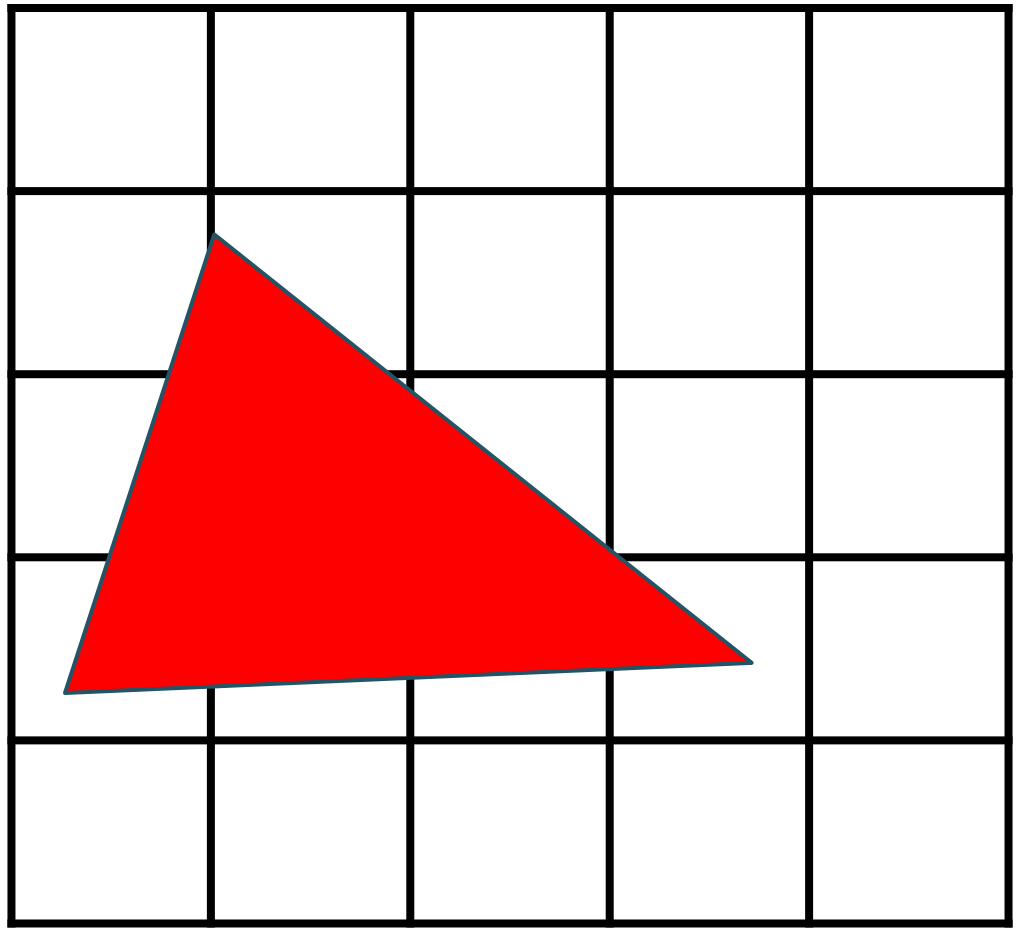


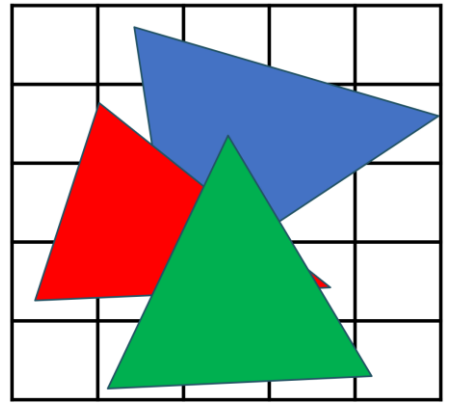
Rasterization



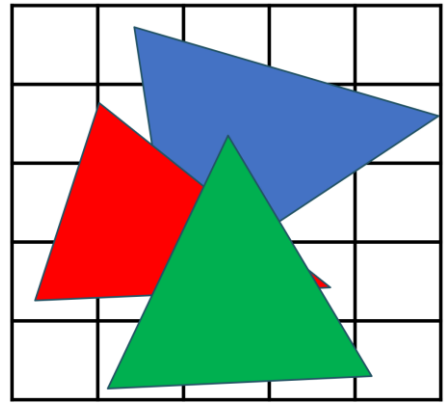


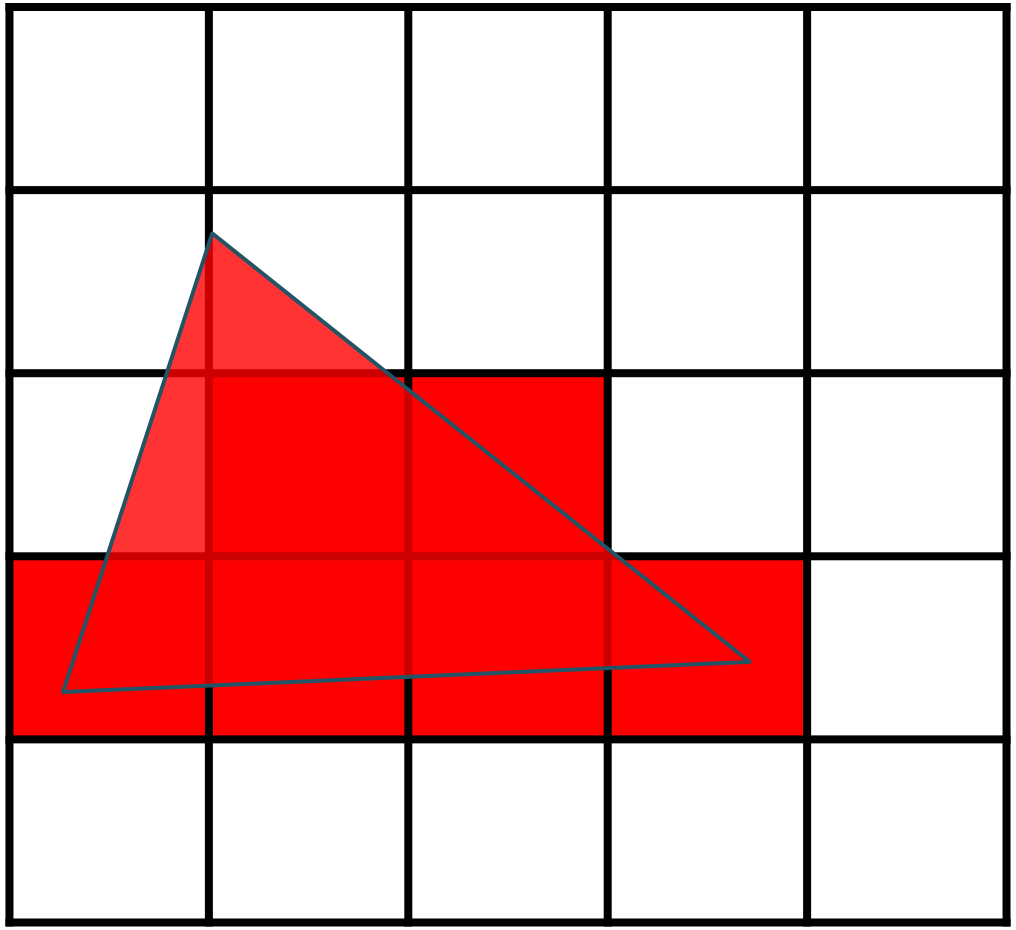
Rasterization



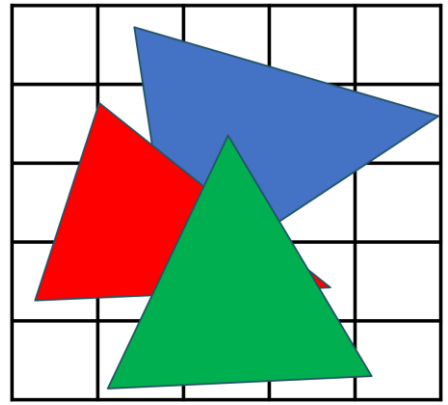


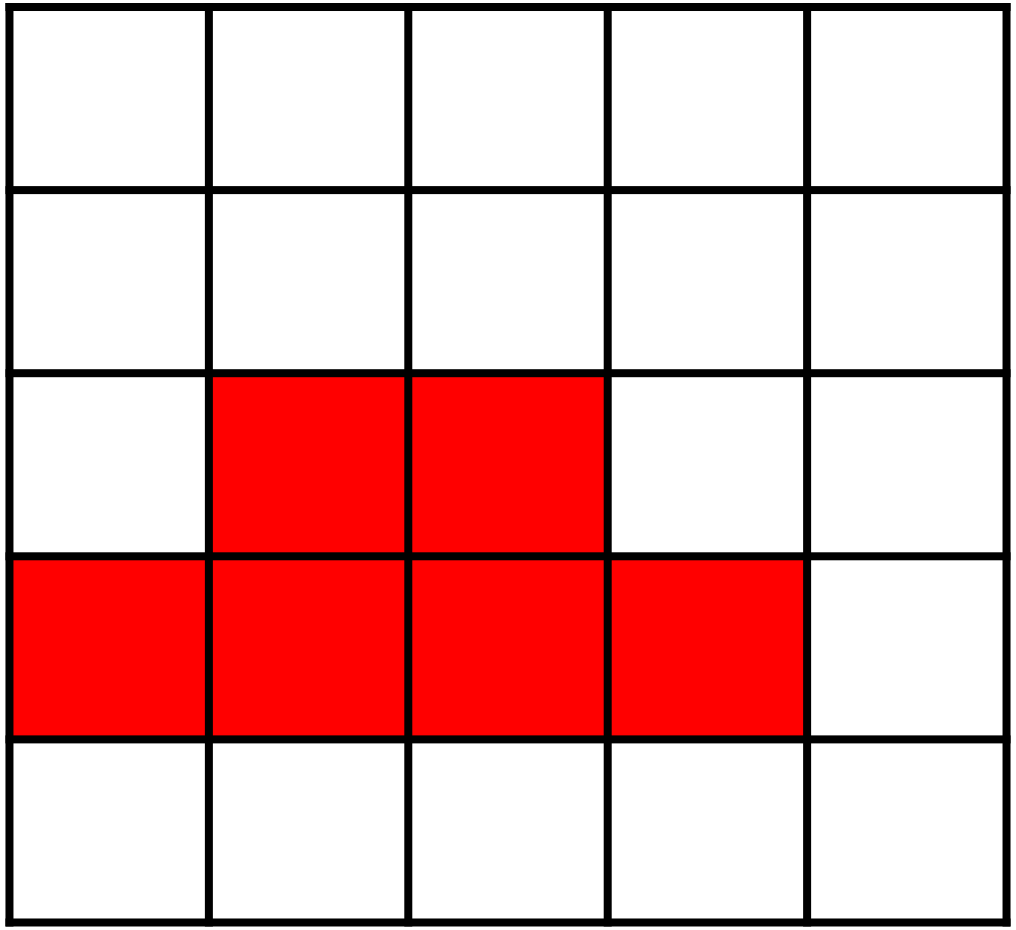
Rasterization



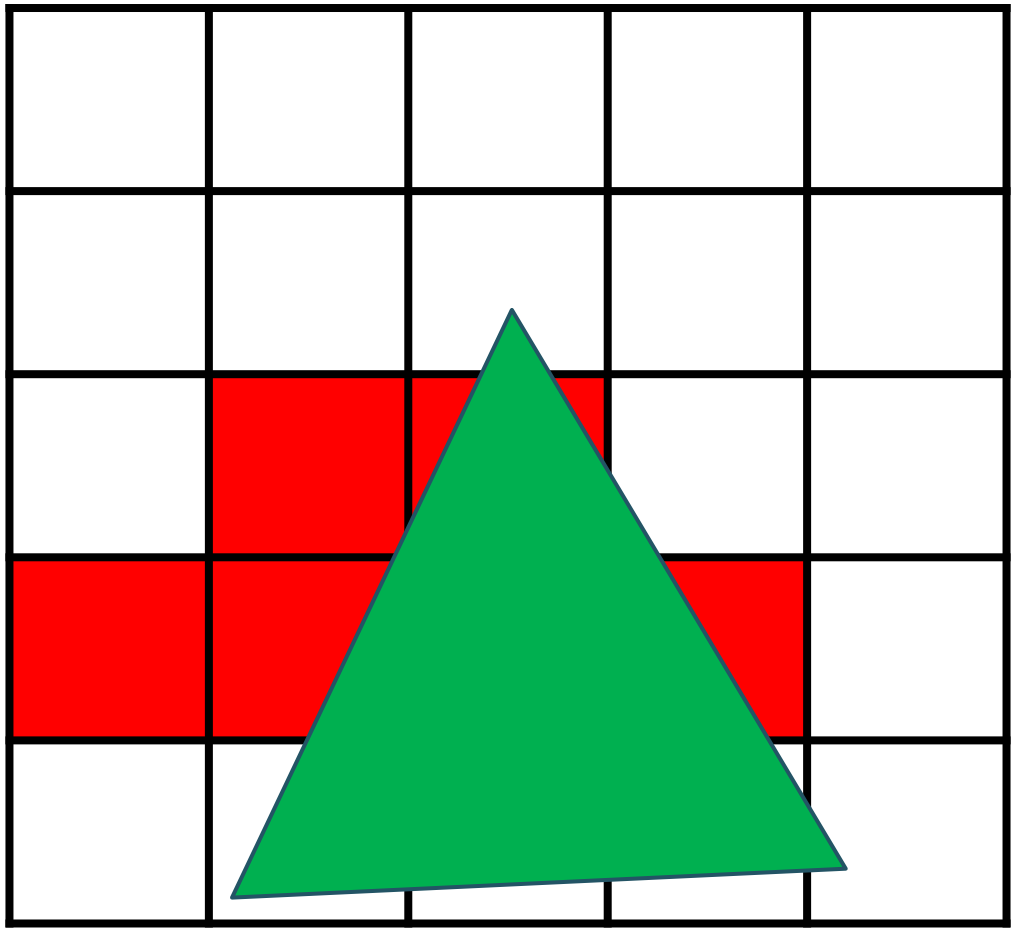


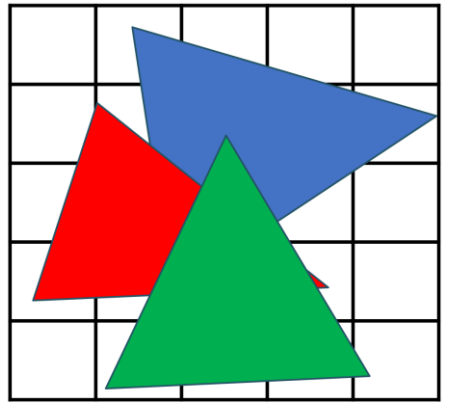
Rasterization



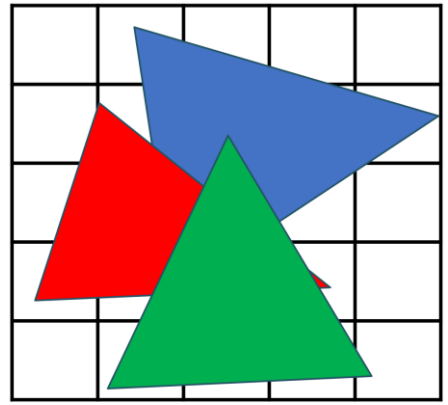


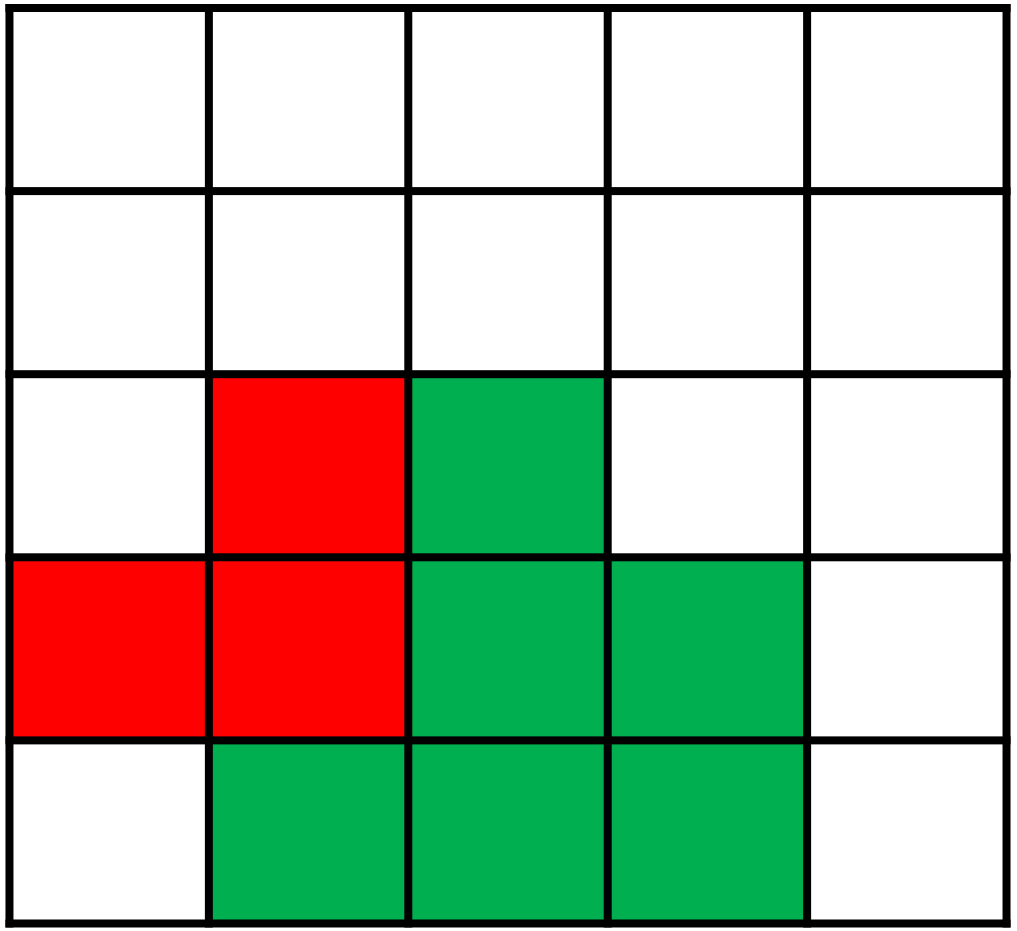
Rasterization



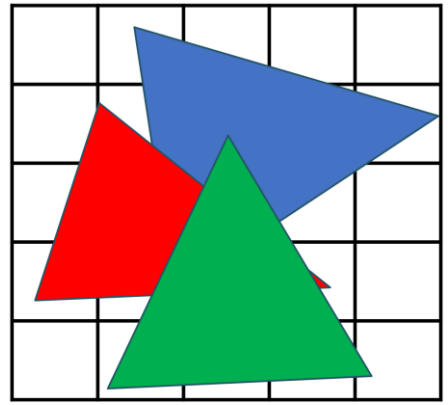


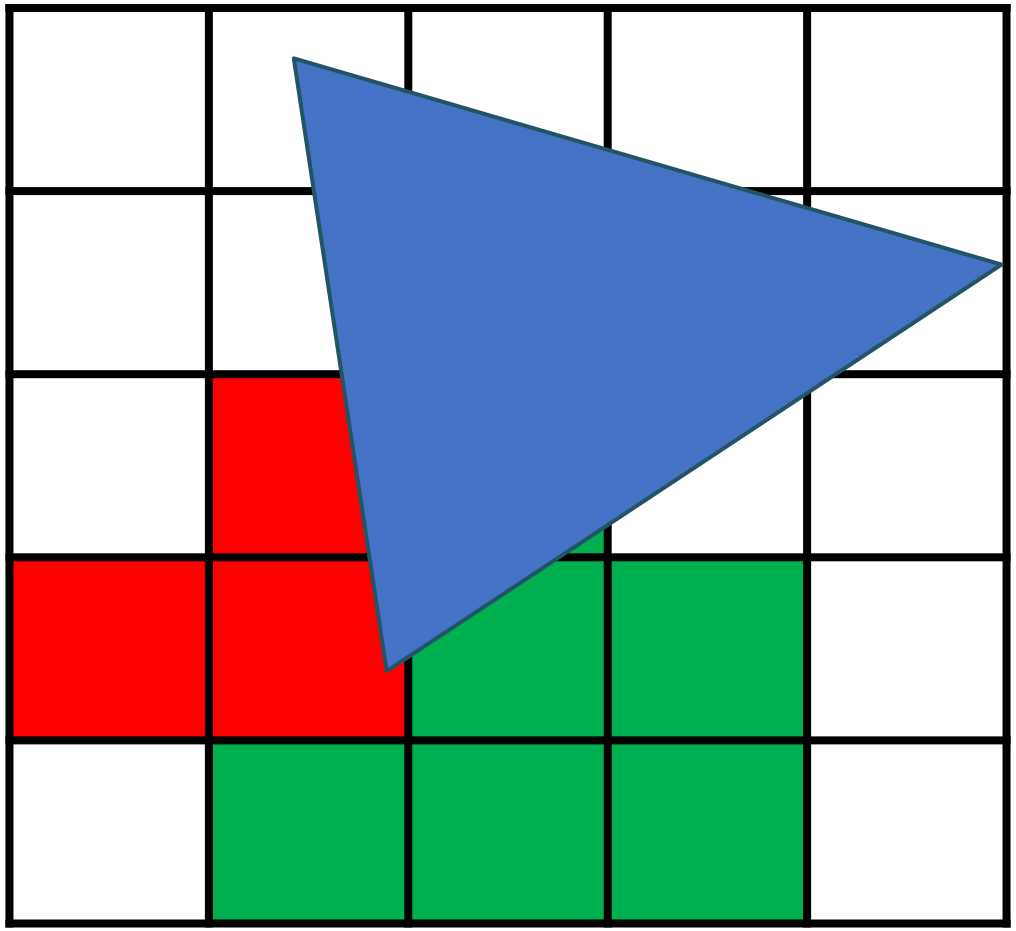
Rasterization



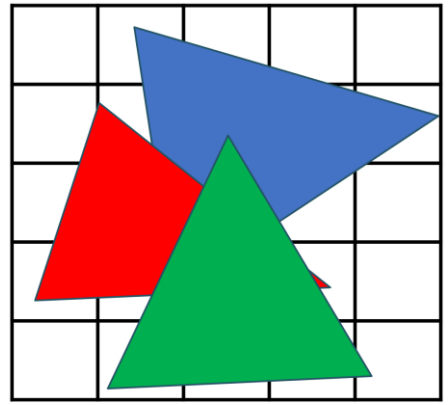


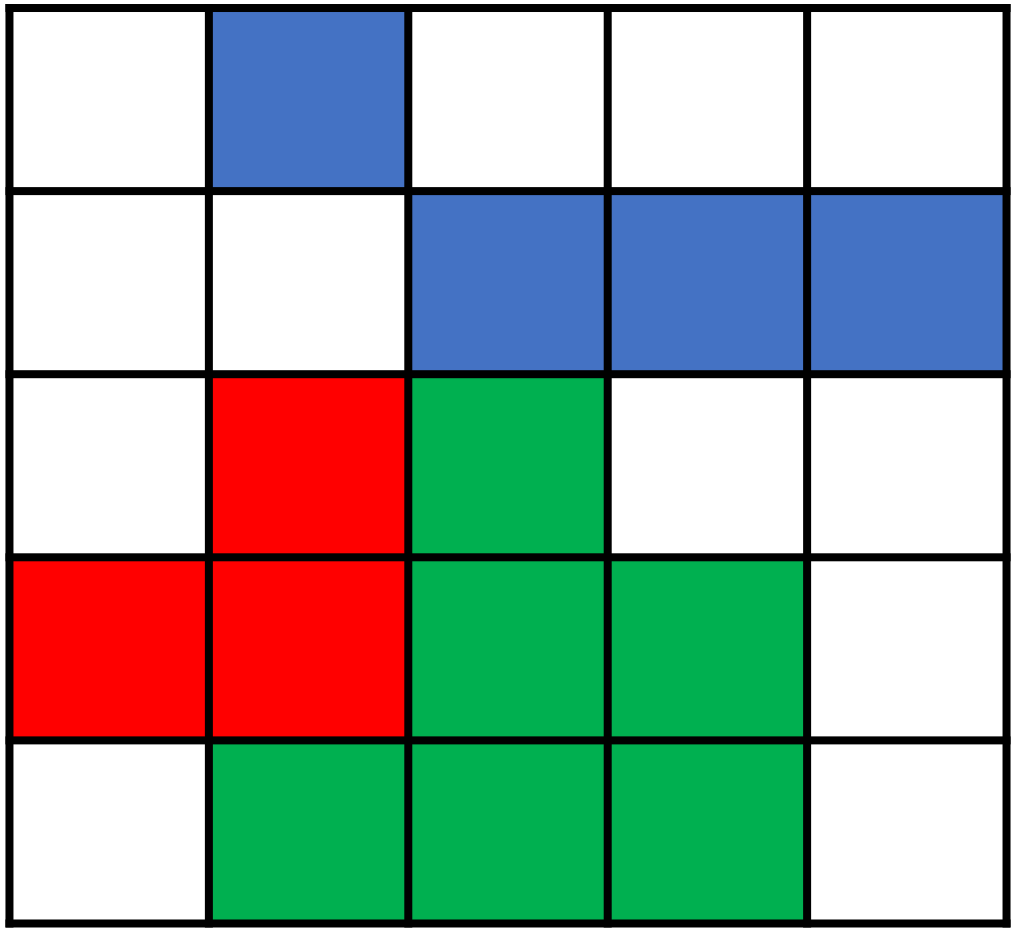
Rasterization



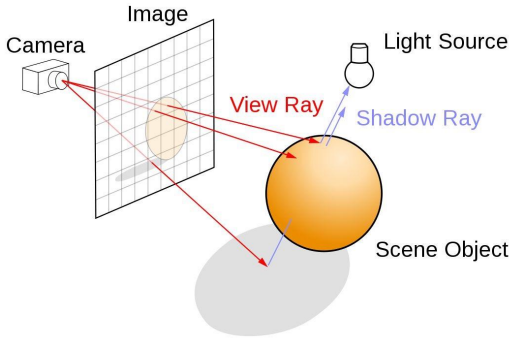


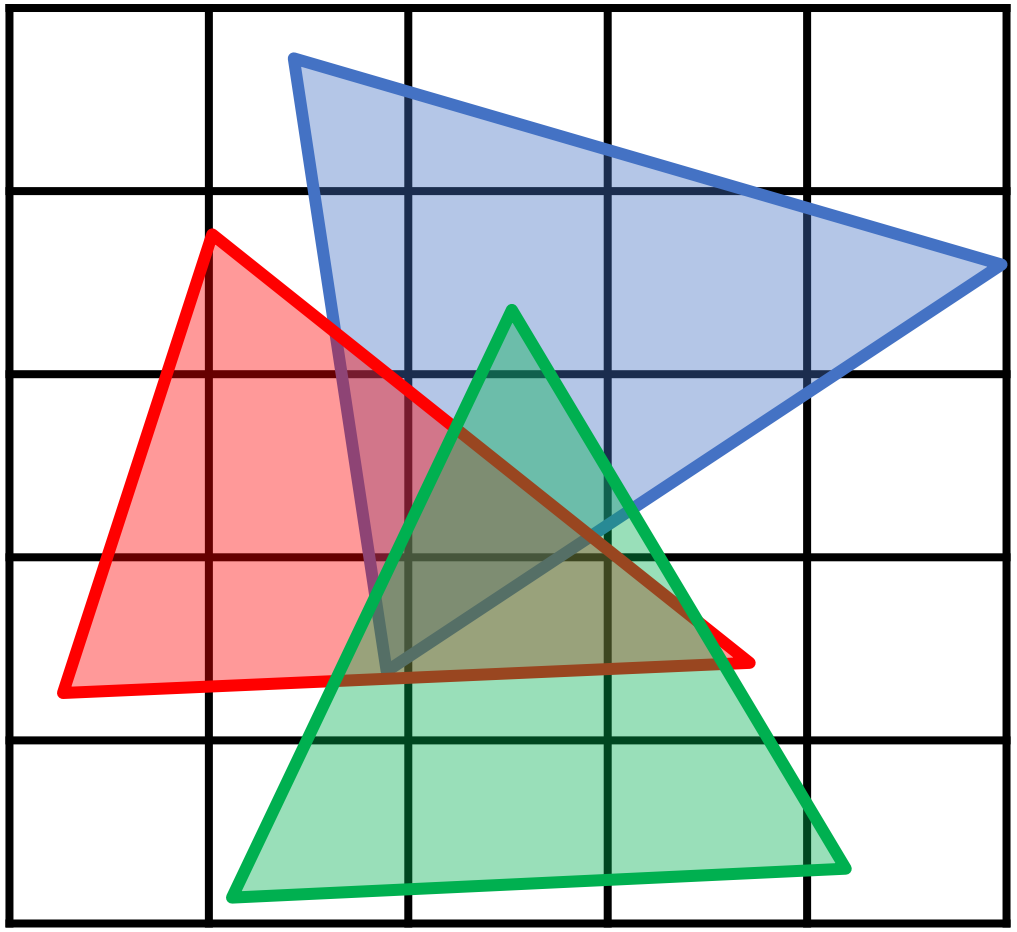
Rasterization

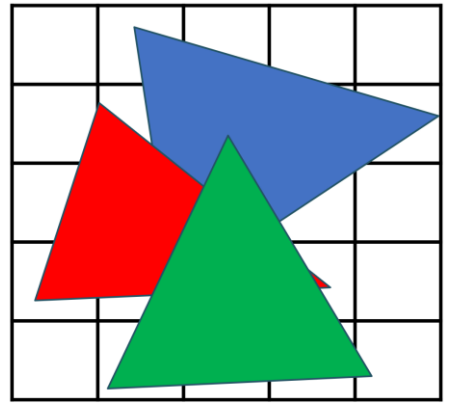




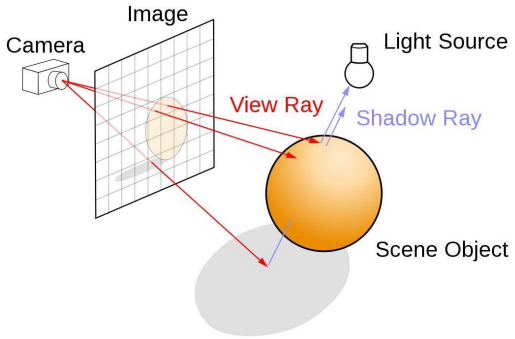
Ray Tracing

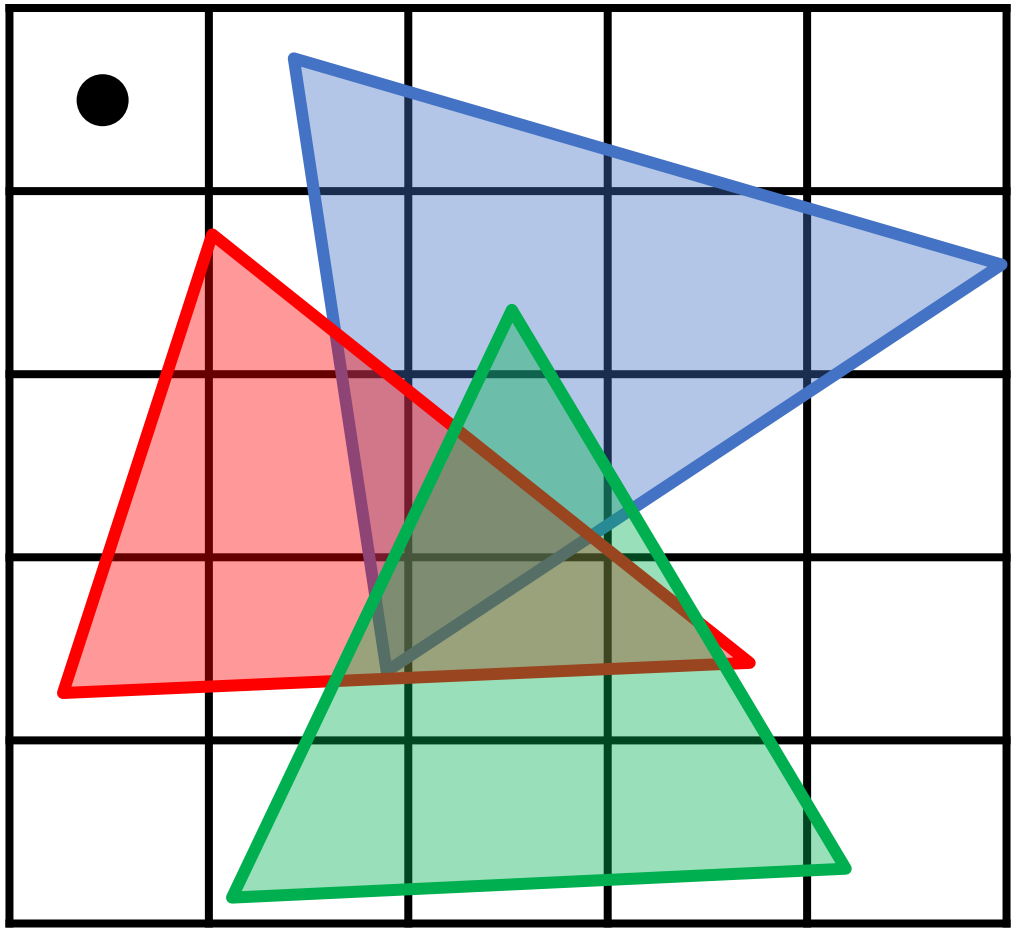


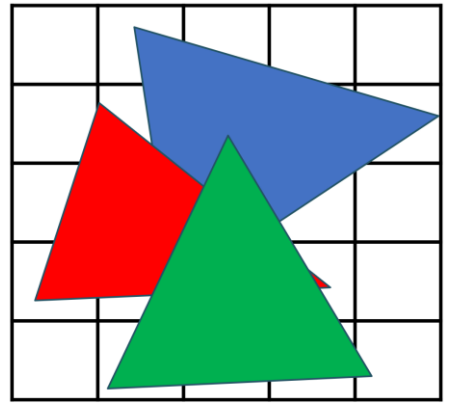




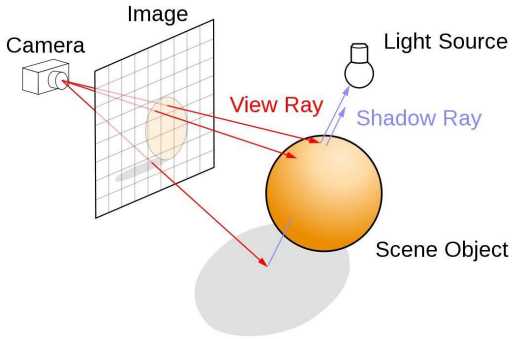
Ray Tracing

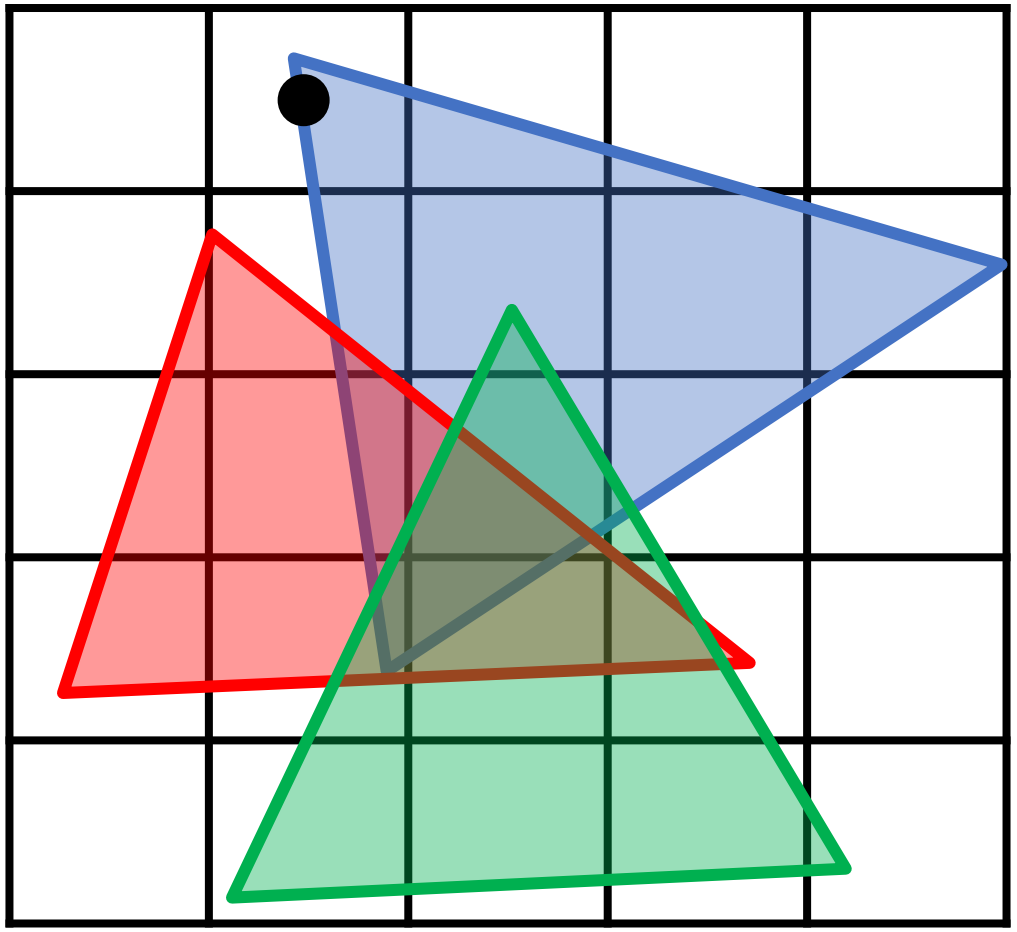


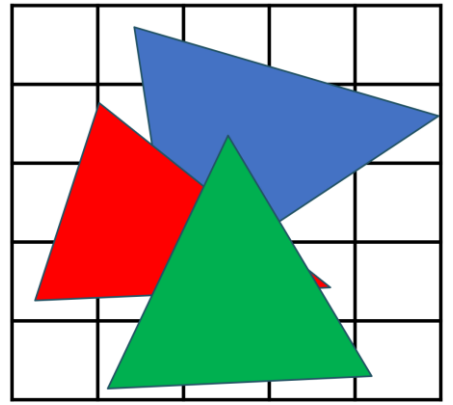




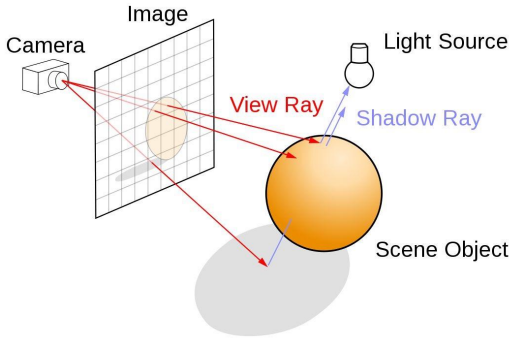
Ray Tracing

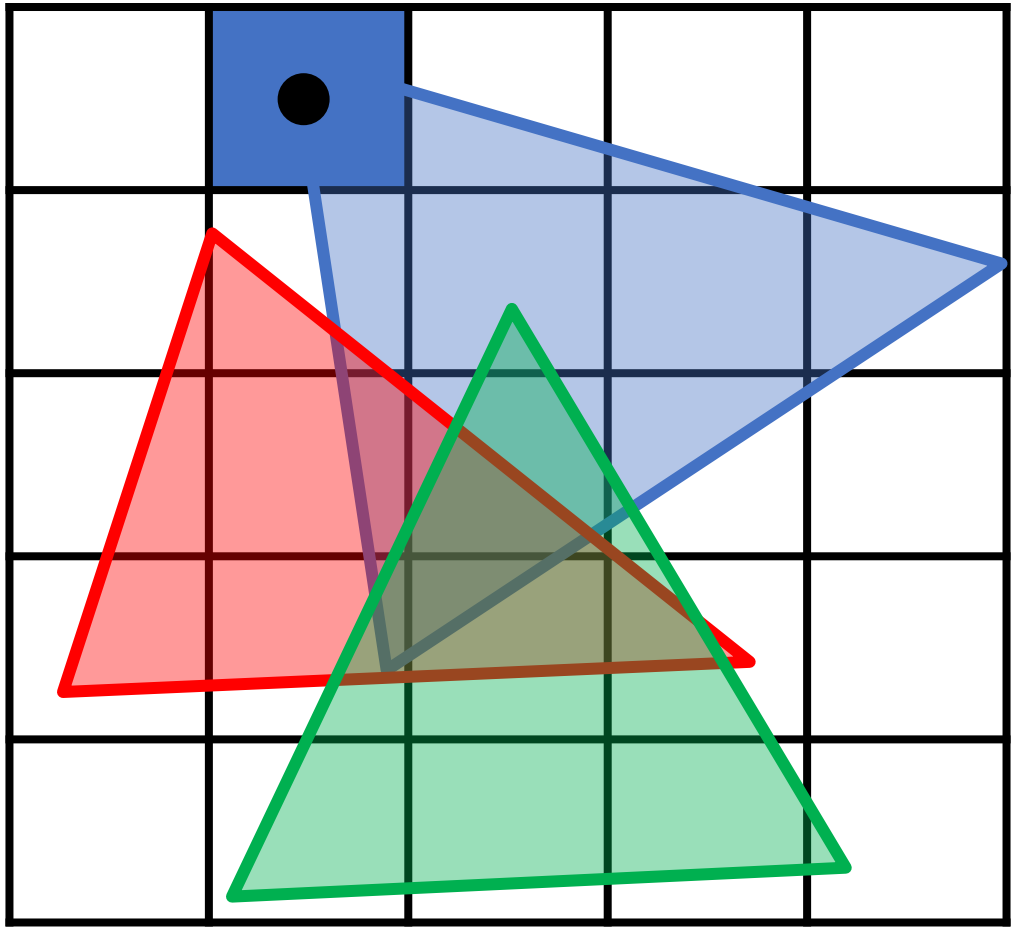


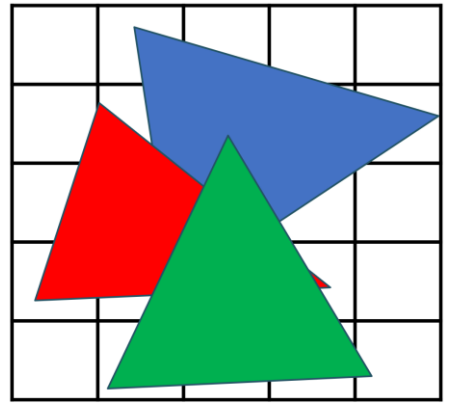




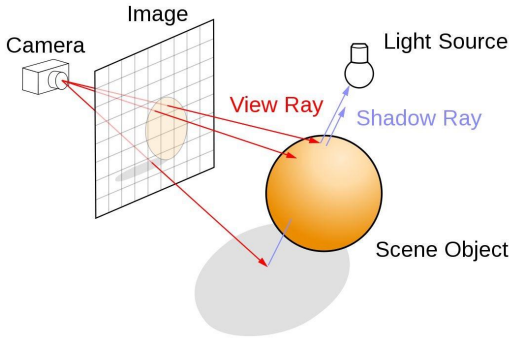
Ray Tracing

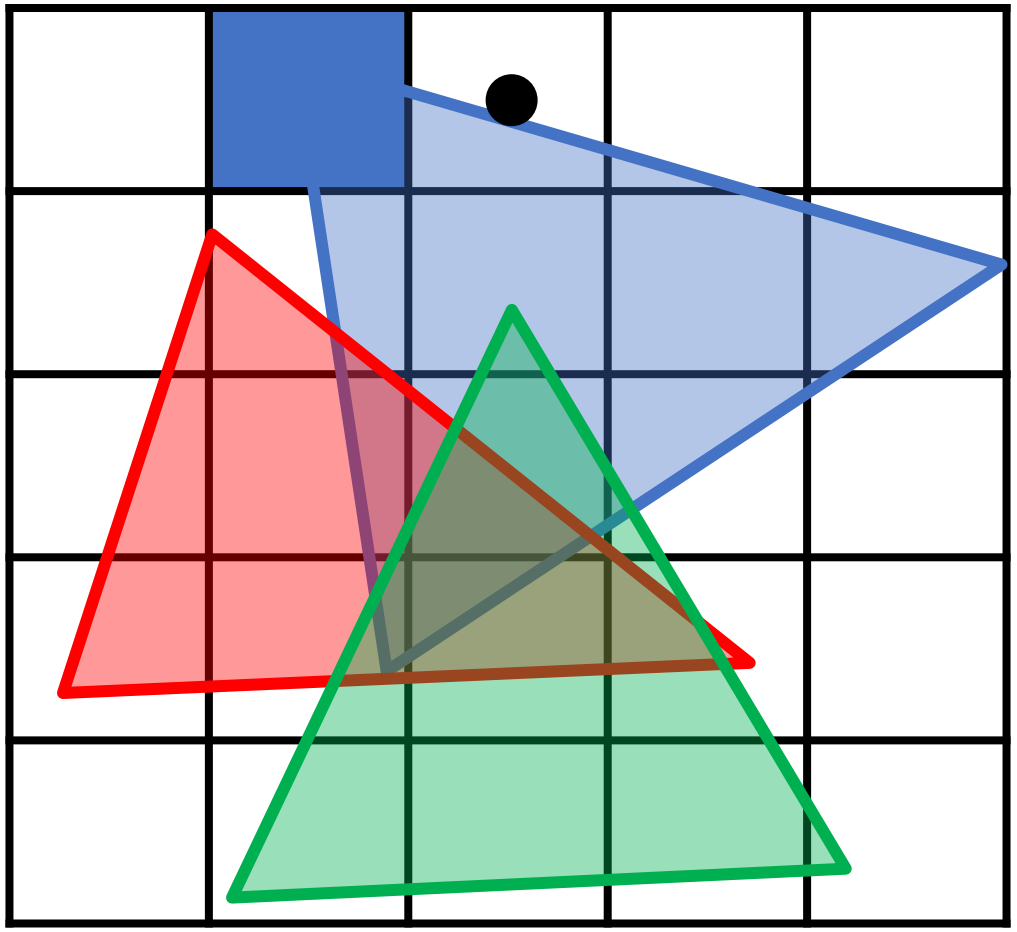


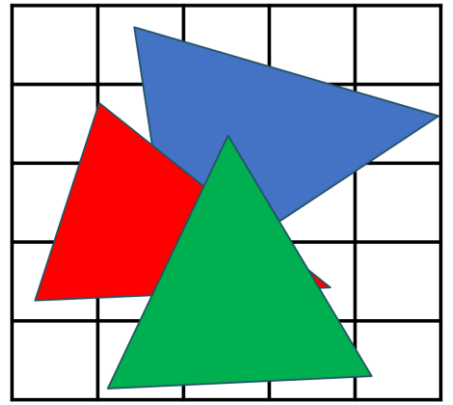




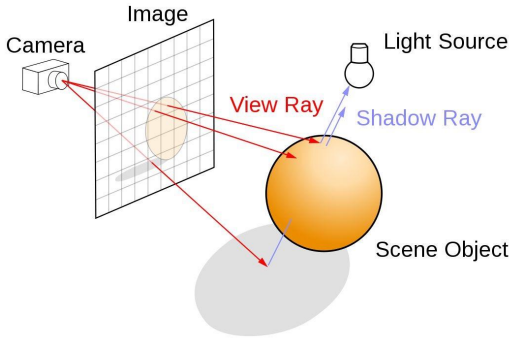
Ray Tracing

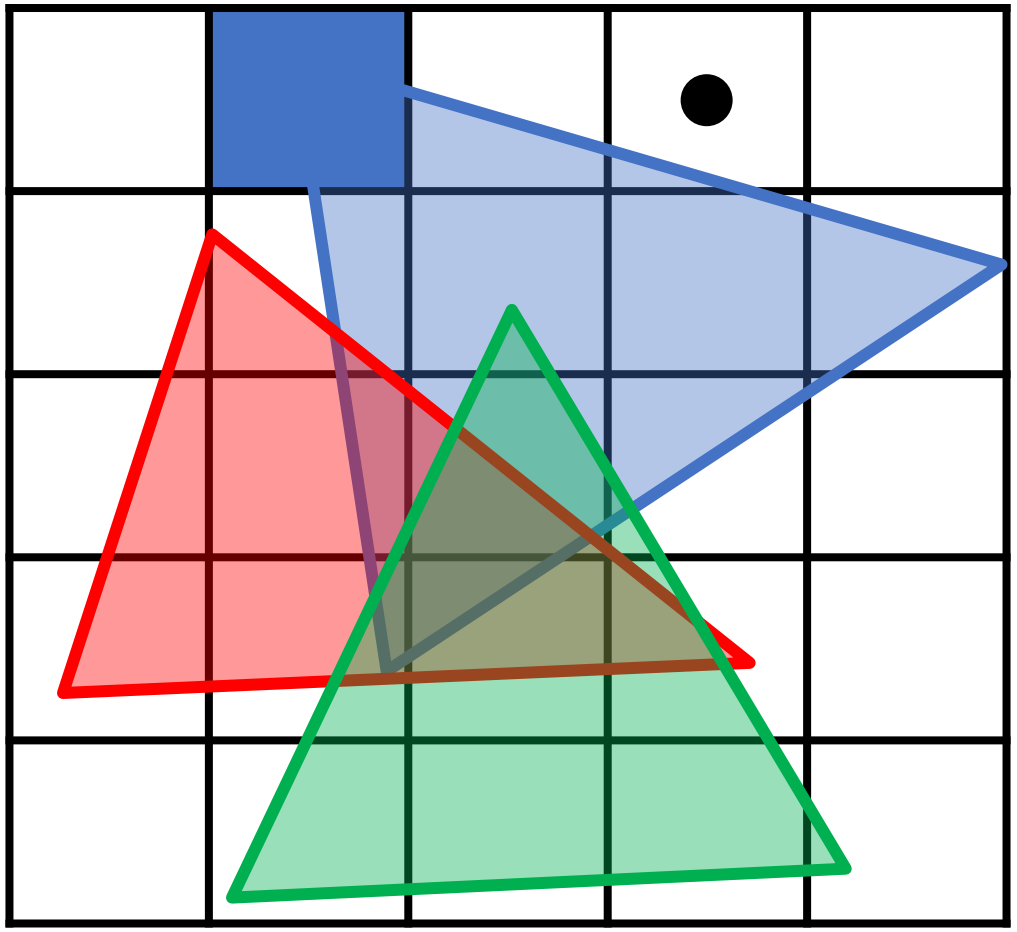


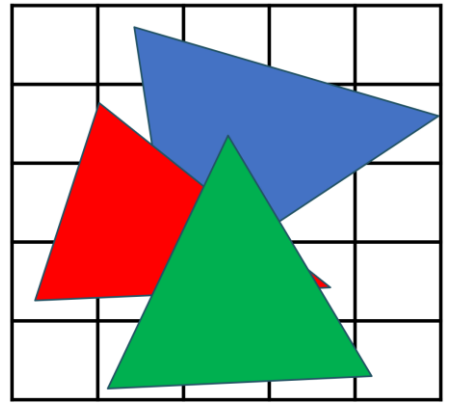




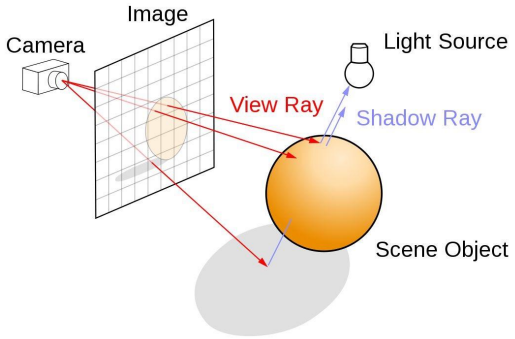
Ray Tracing

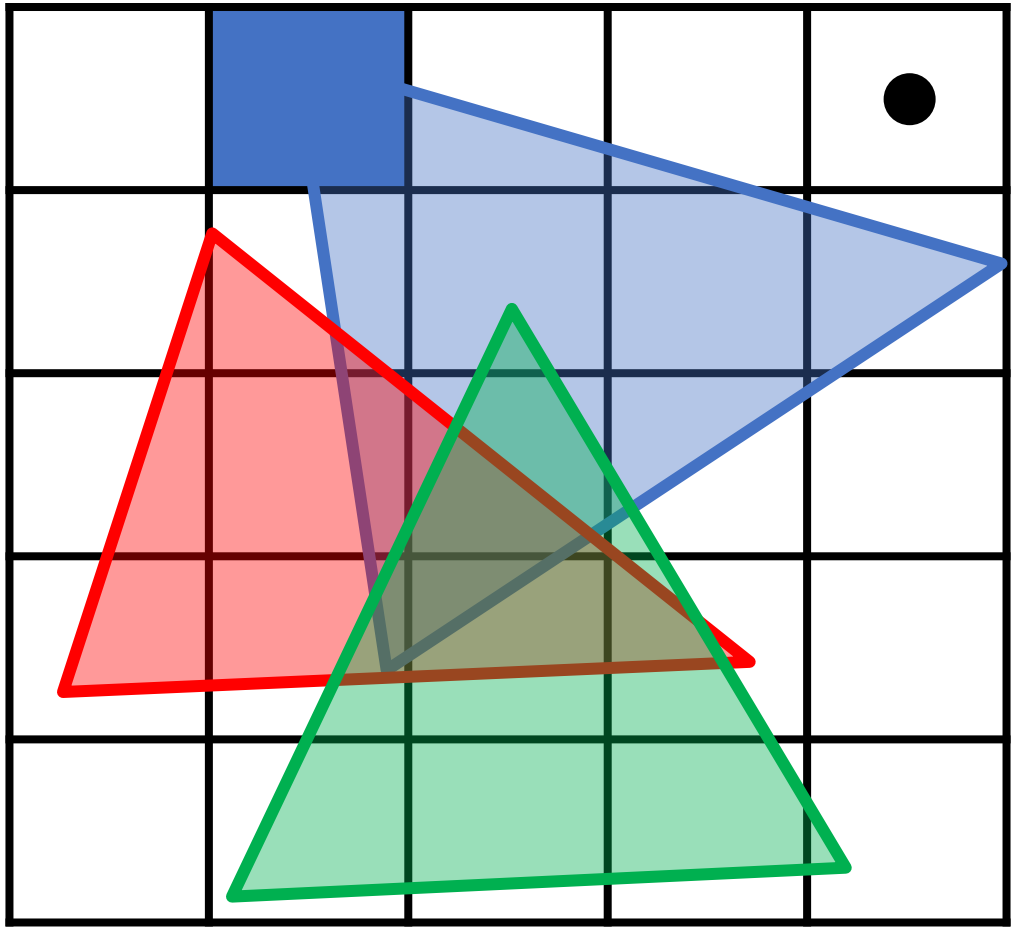


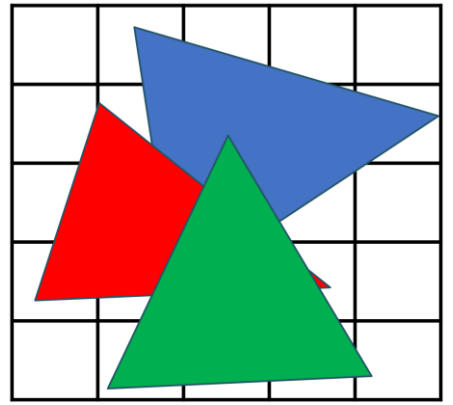




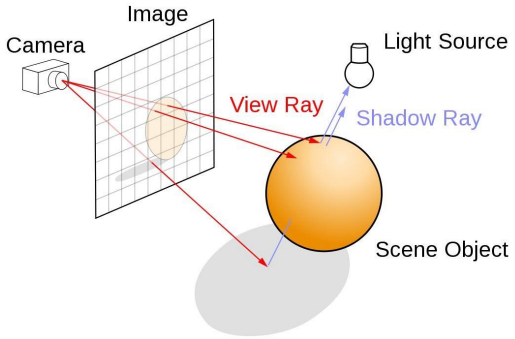
Ray Tracing

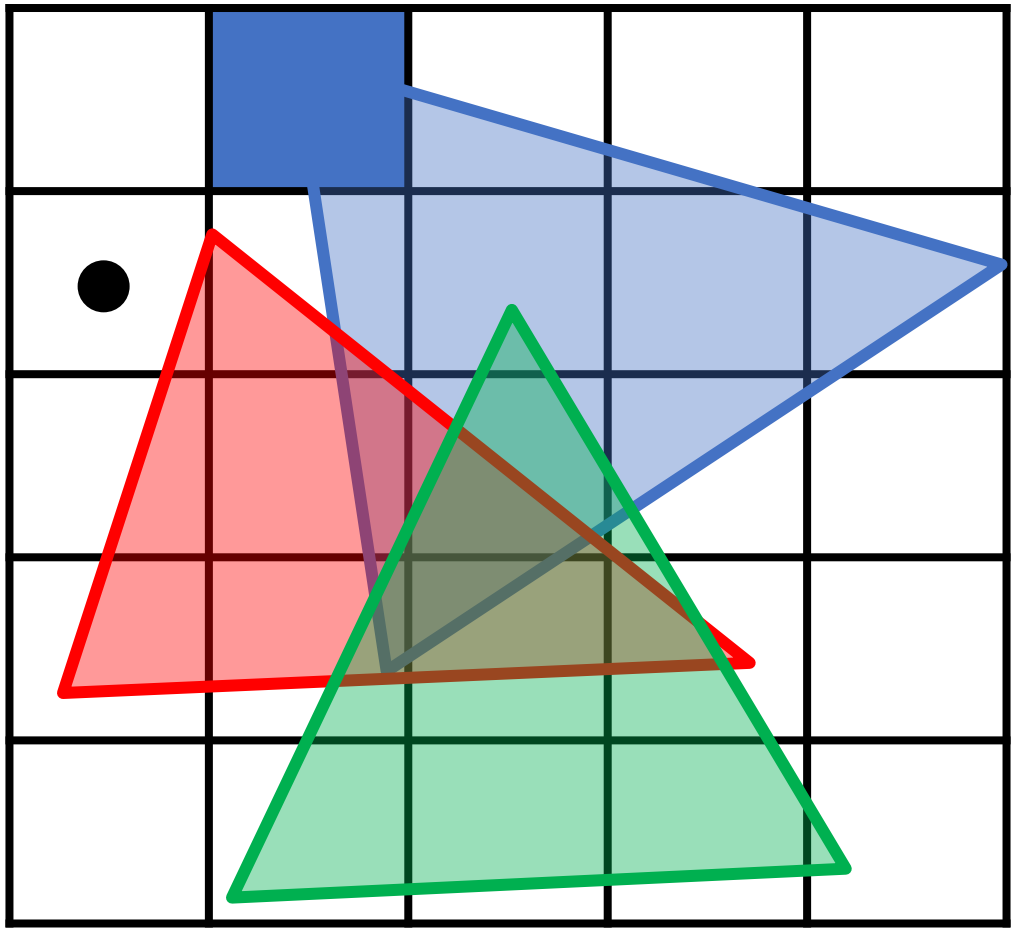


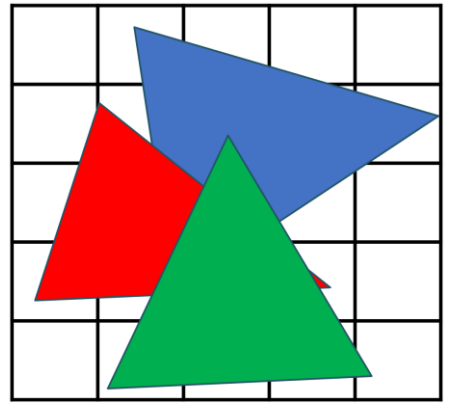




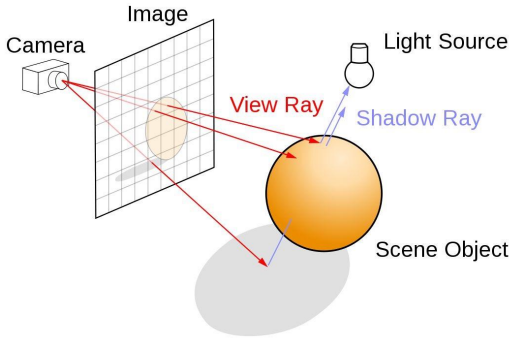
Ray Tracing

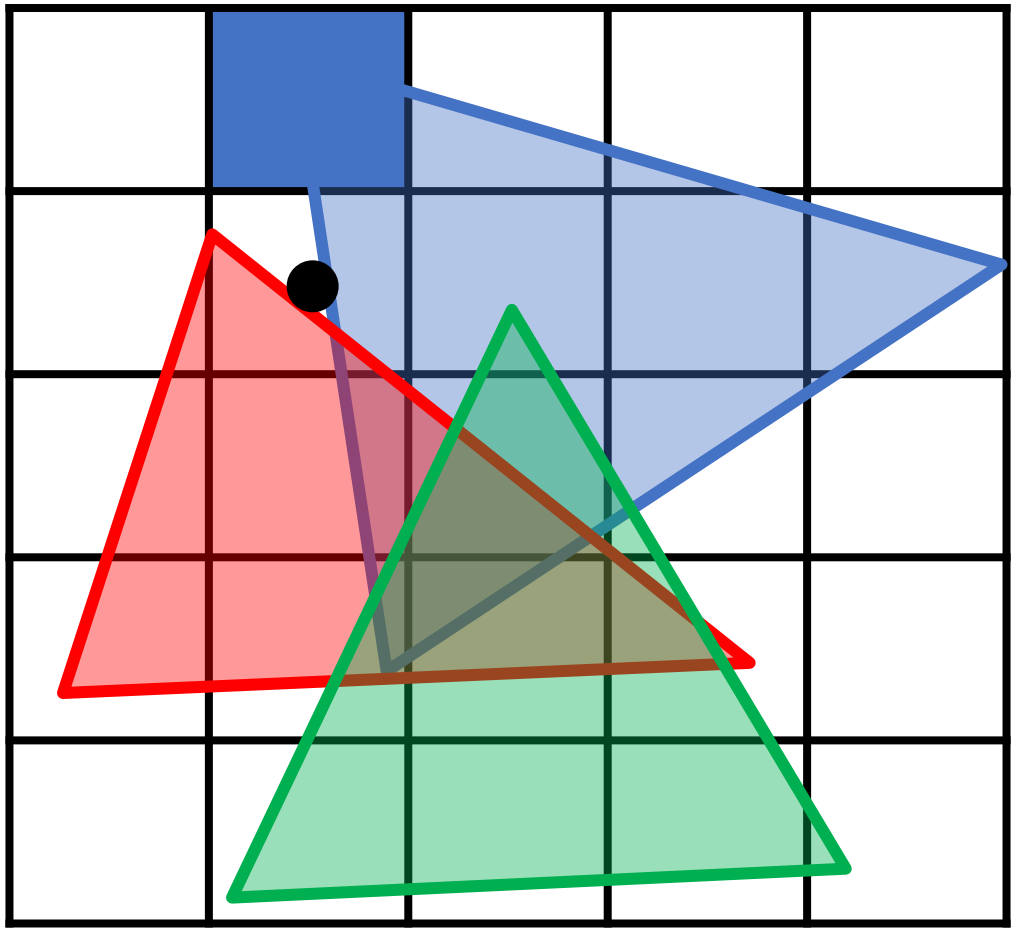


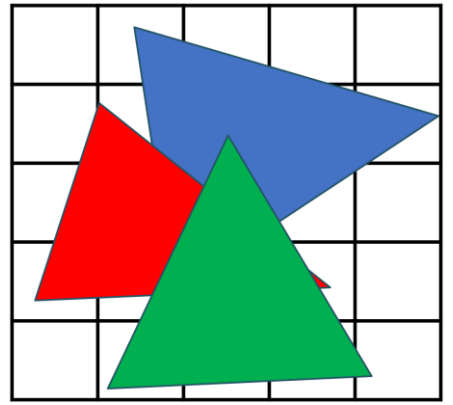




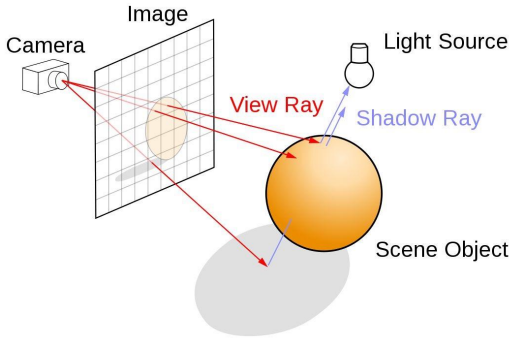
Ray Tracing

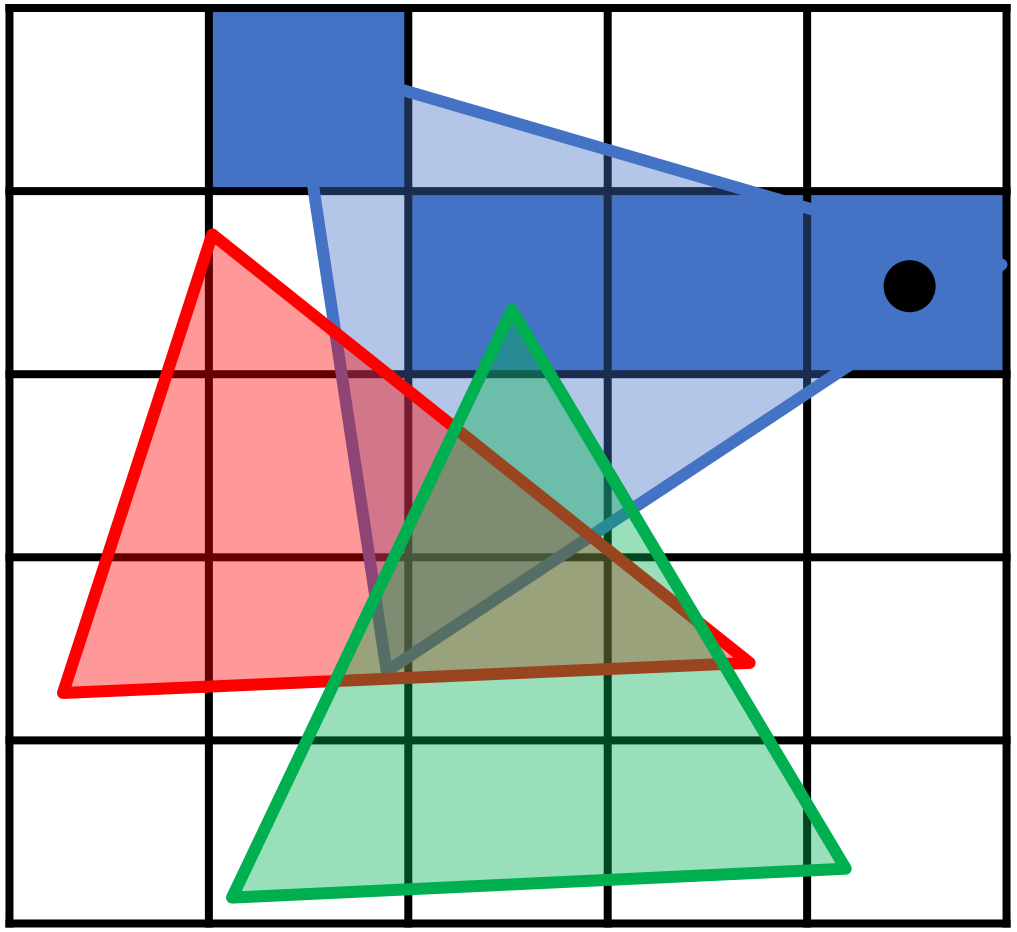


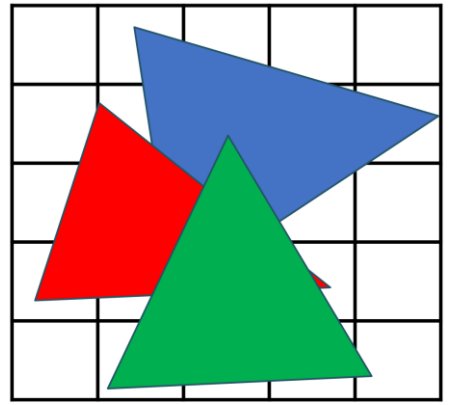




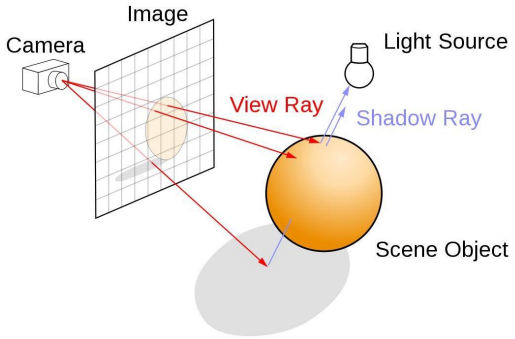
Ray Tracing

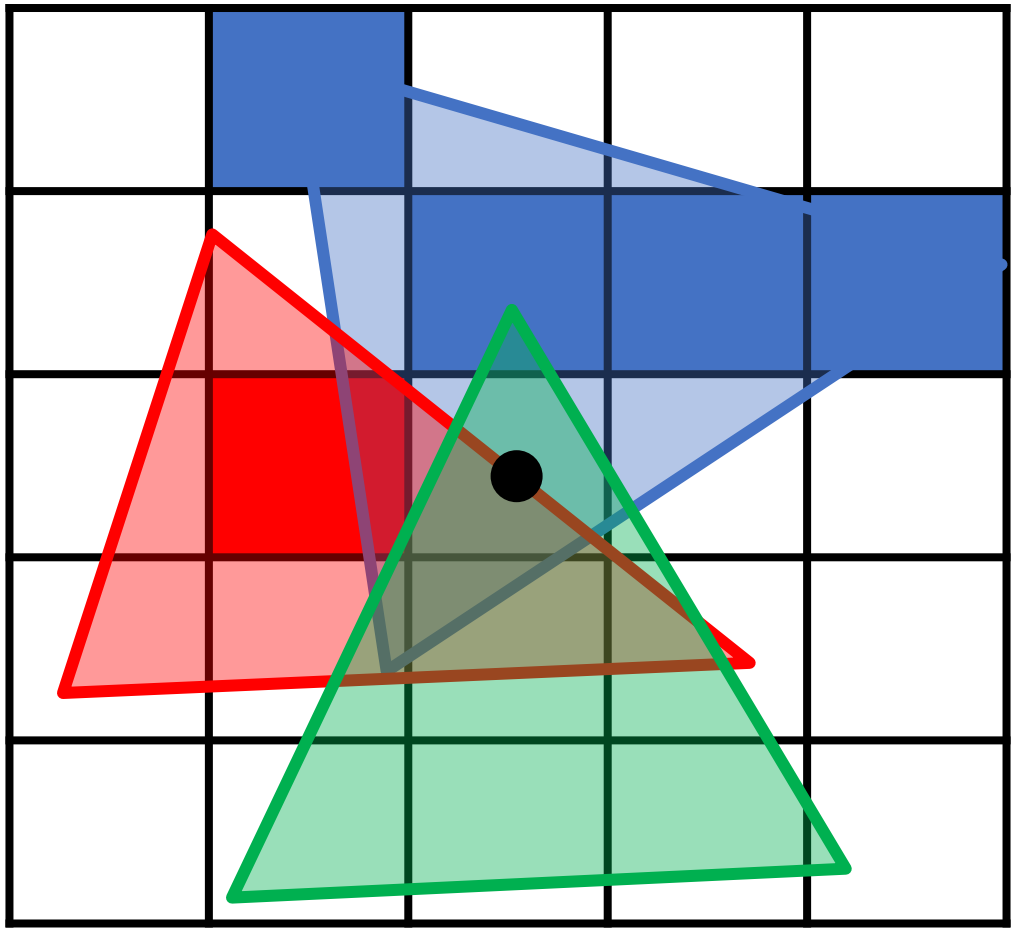


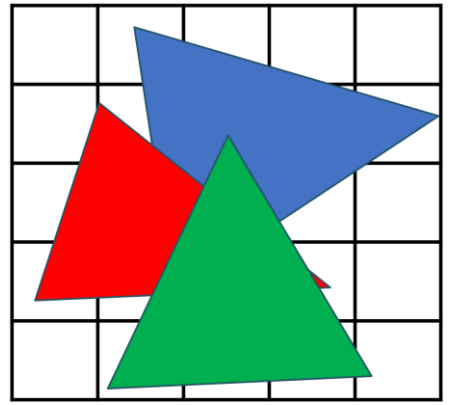




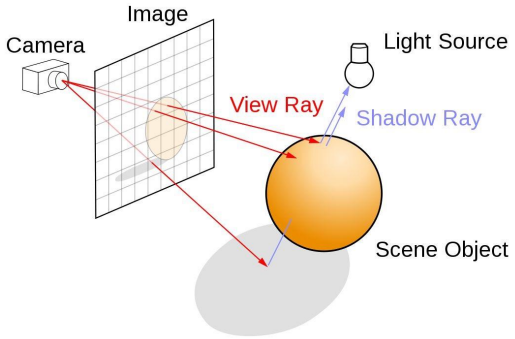
Ray Tracing

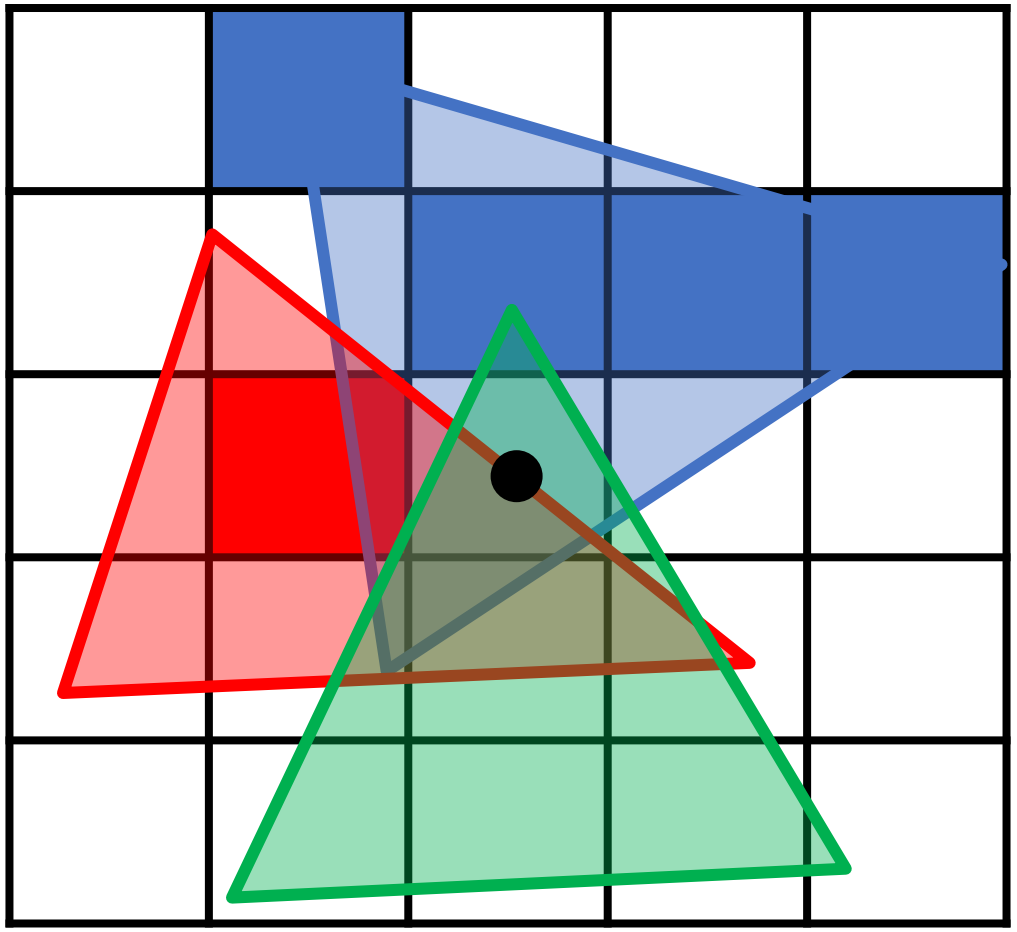


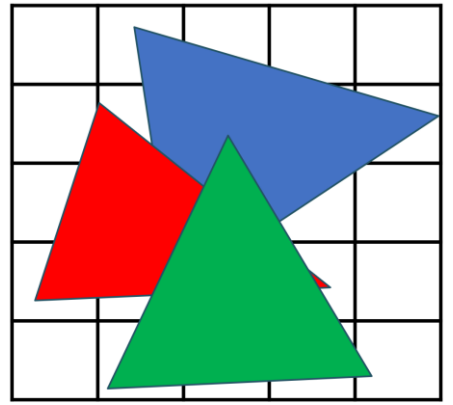




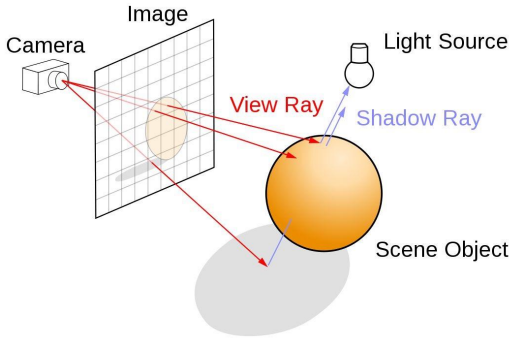
Ray Tracing

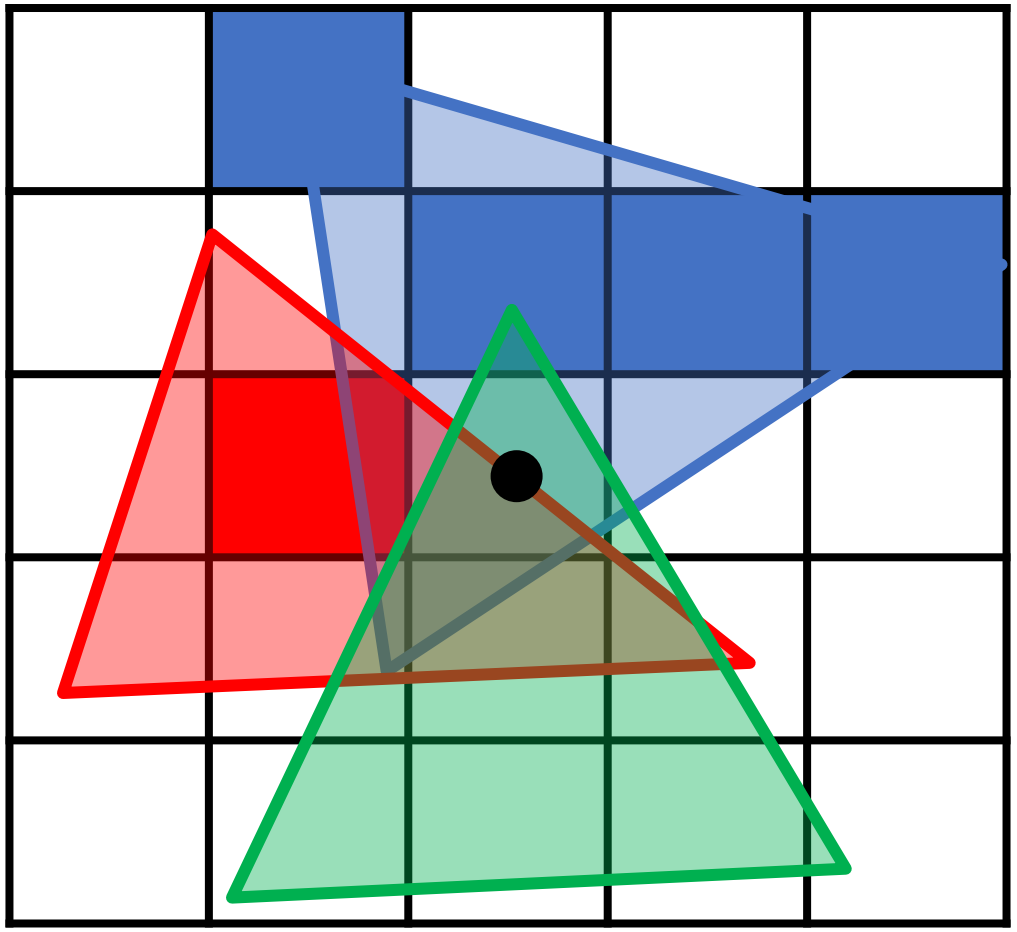


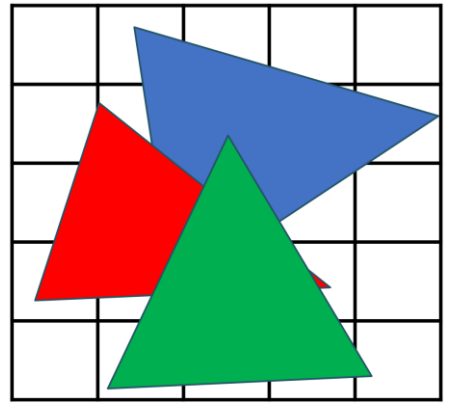




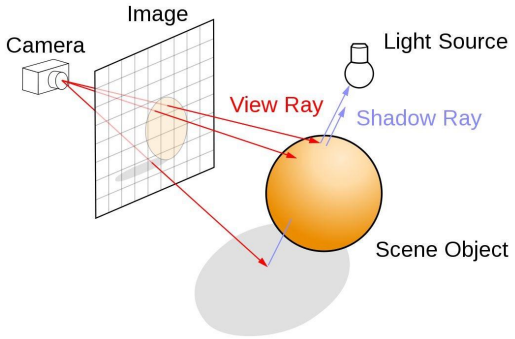
Ray Tracing

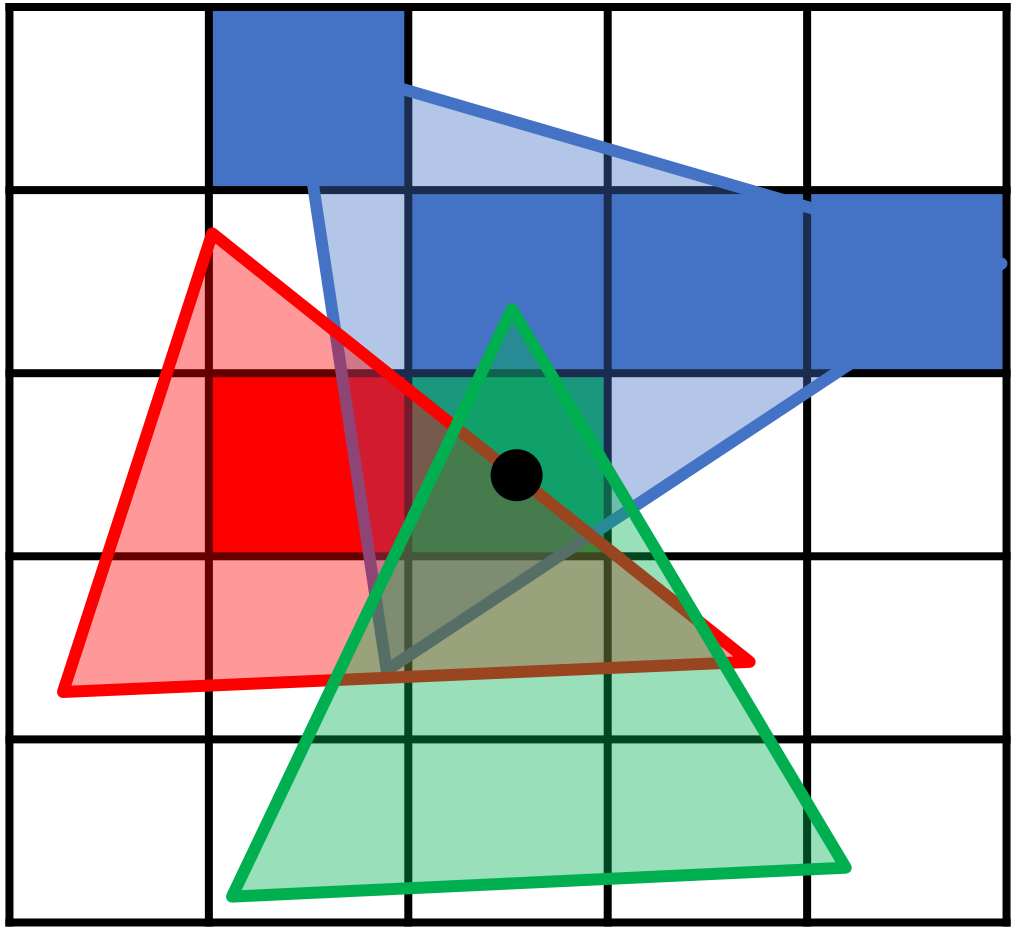


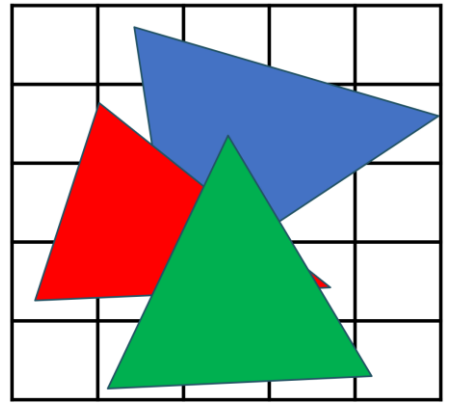




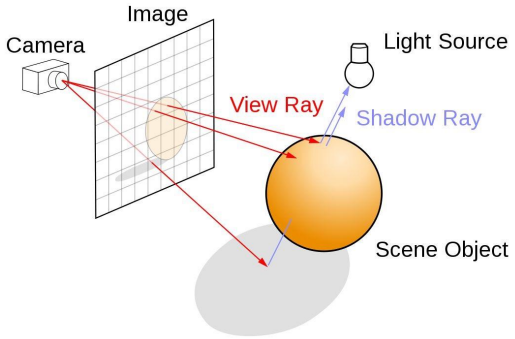
Ray Tracing

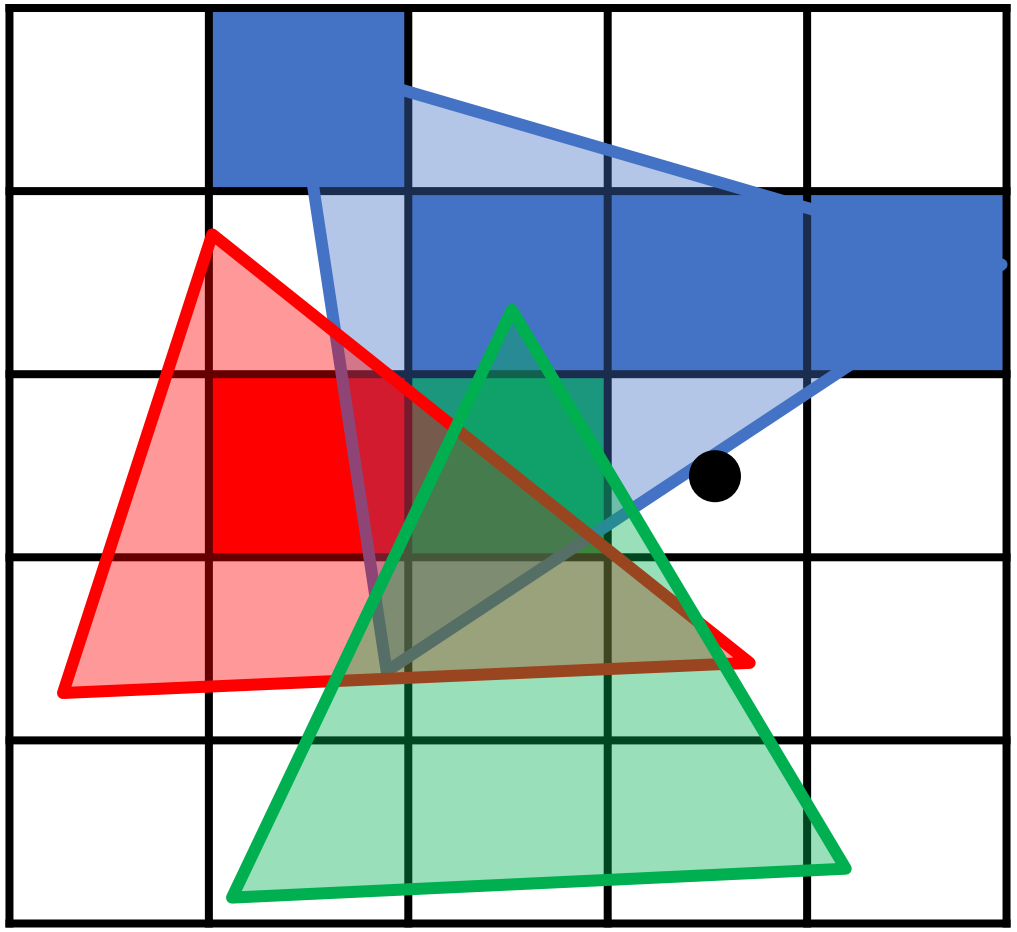


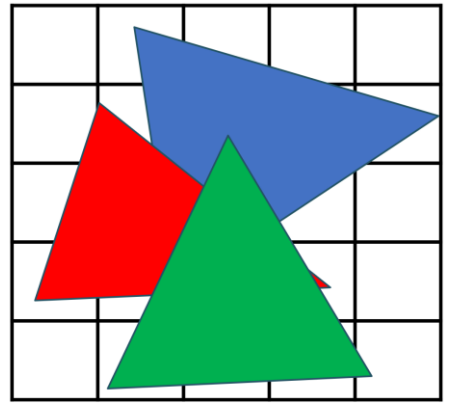




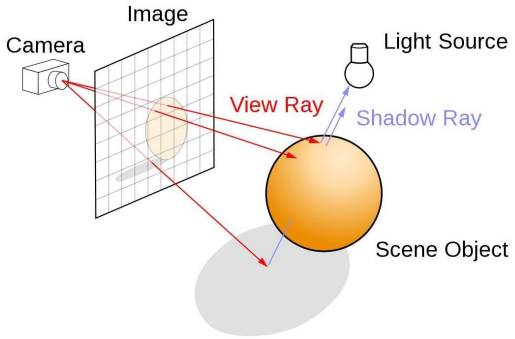
Ray Tracing

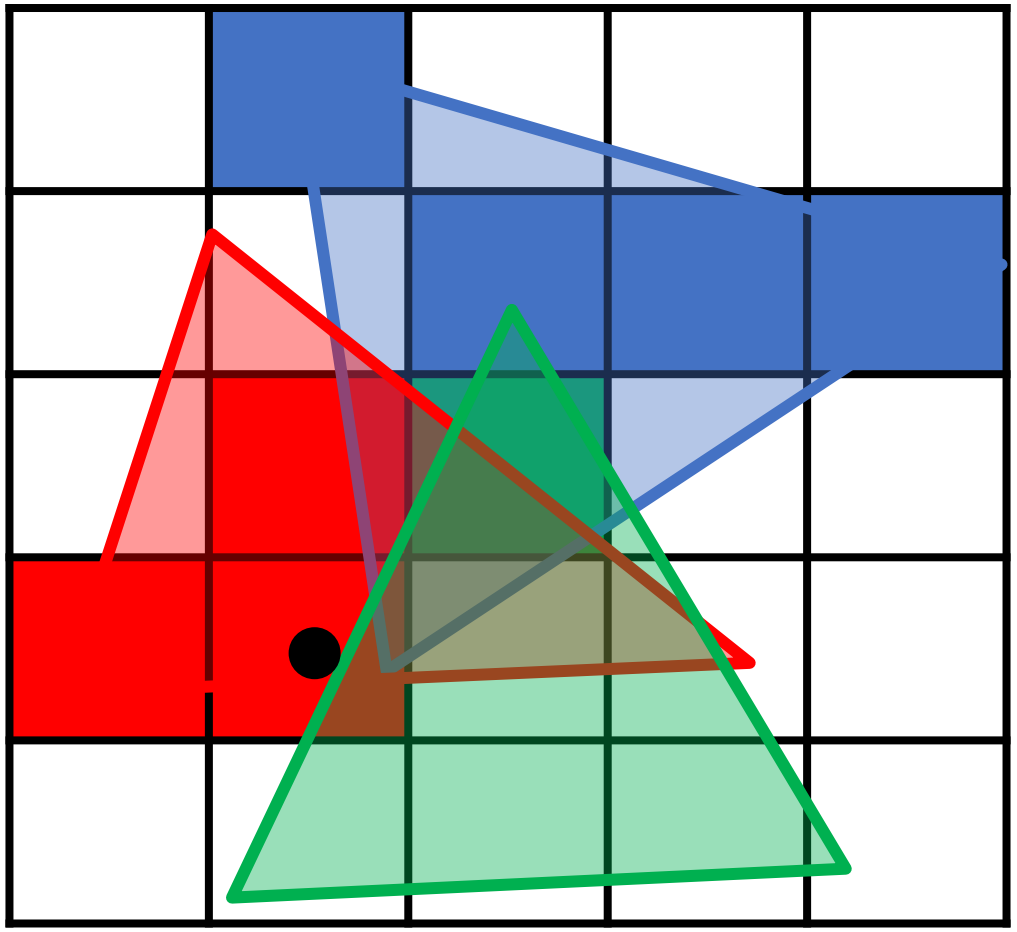


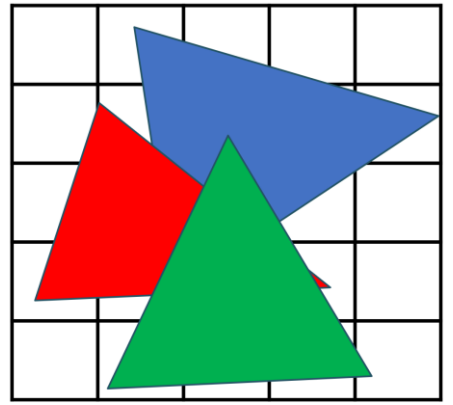




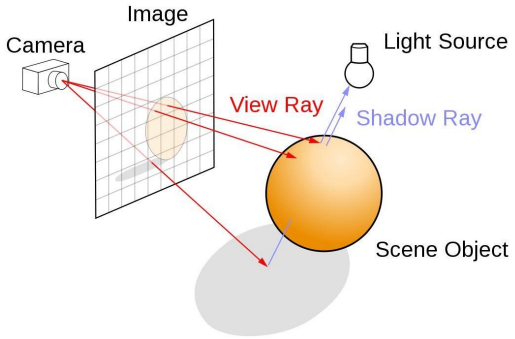
Ray Tracing

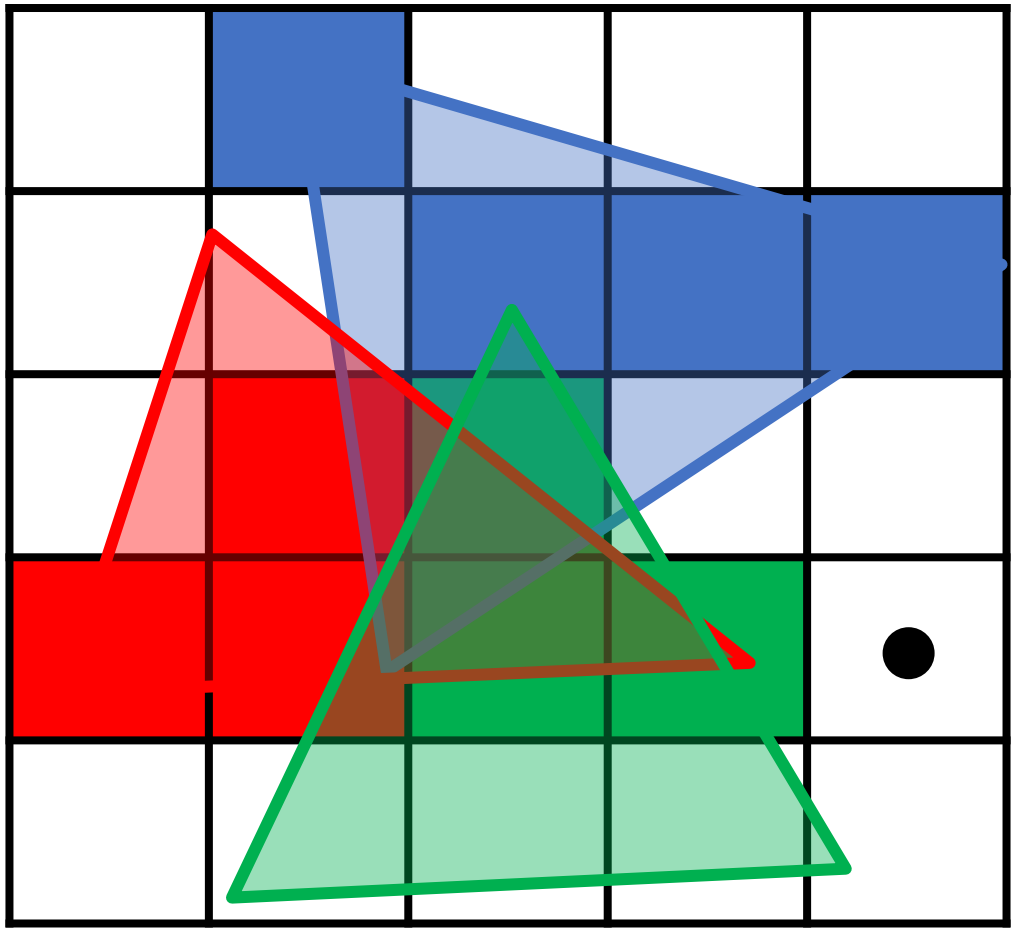


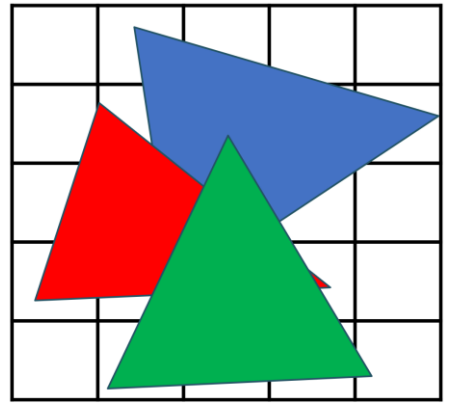




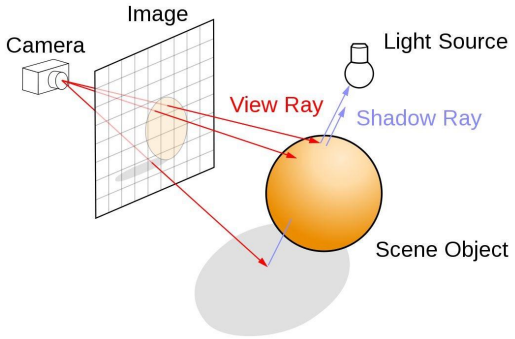
Ray Tracing

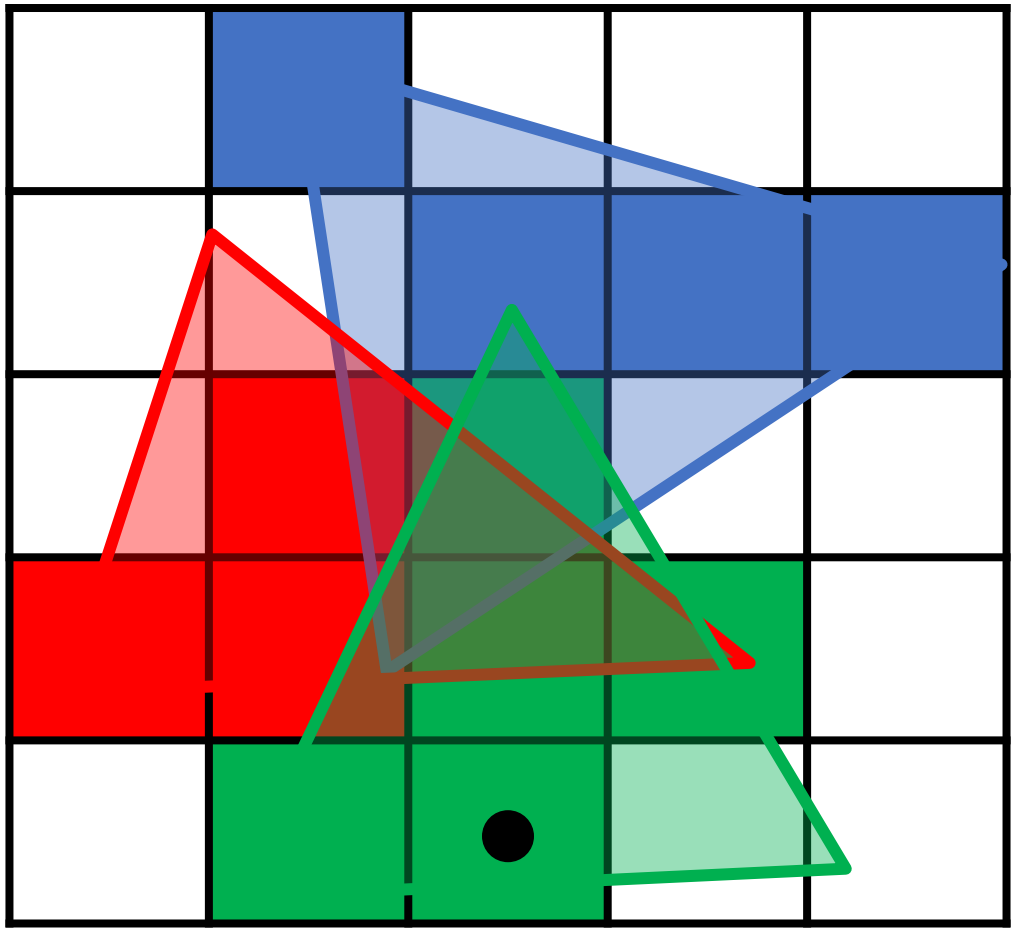


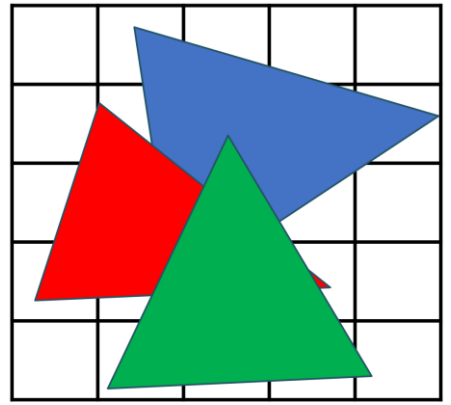




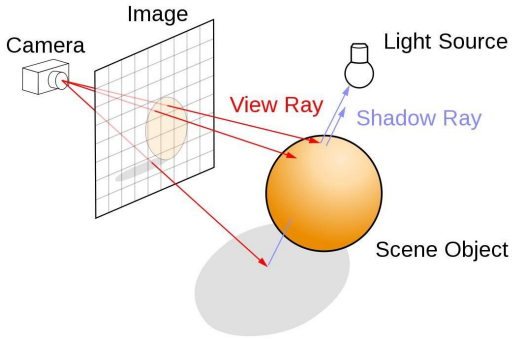
Ray Tracing

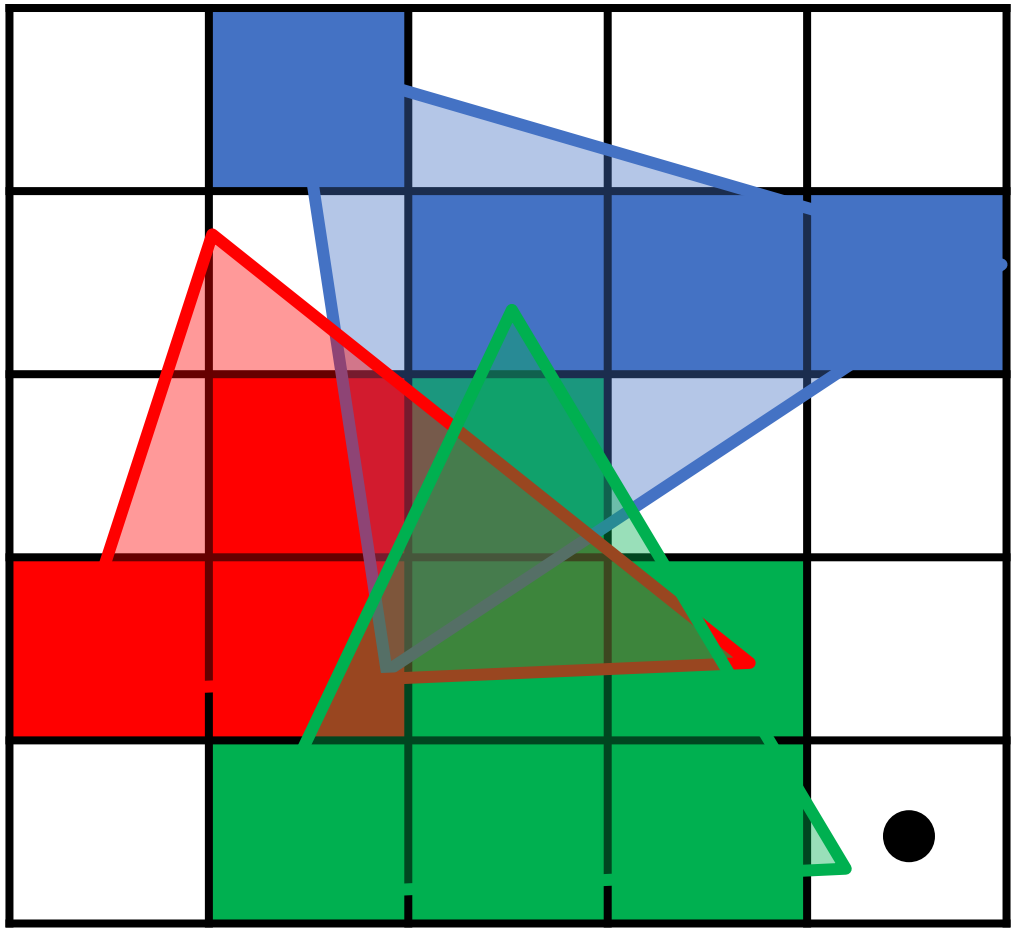


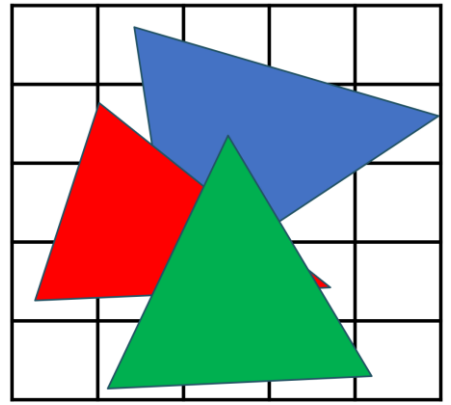




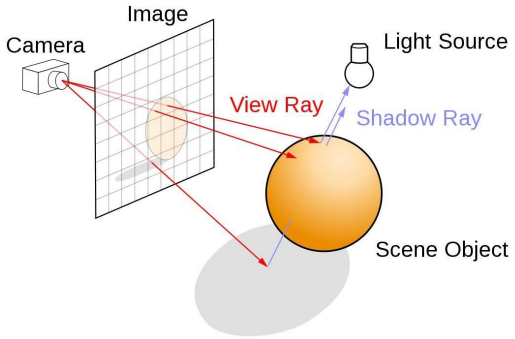
Ray Tracing

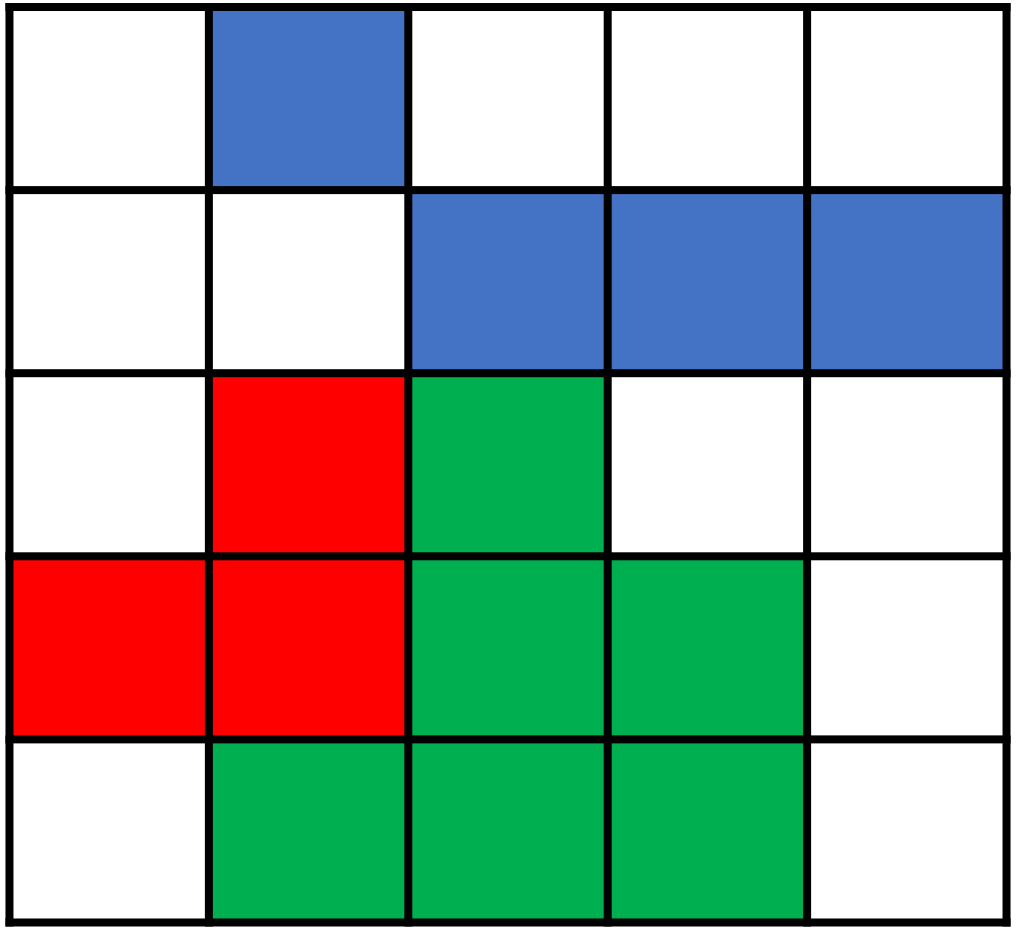


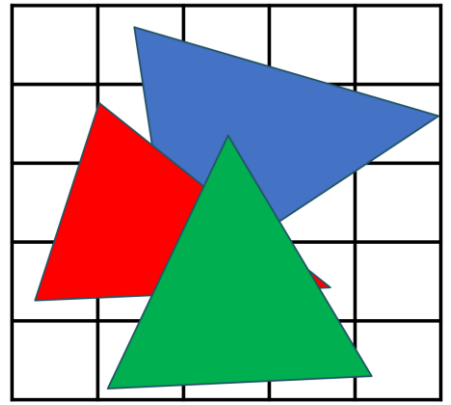




Ray Tracing







In Perspective: Rasterization vs Ray Tracing

Rasterization Loop

```
for t in triangles:
    for p in pixels:
        if t covers p:
            paint p with t if on top
```

By Z-buffer

save color if closer

Ray Tracing Loop

```
for each pixel (ray r)
    for each triangle t:
        if r intersects t:
            paint p with t.color()
```

Ray Casting

Apple, 1968



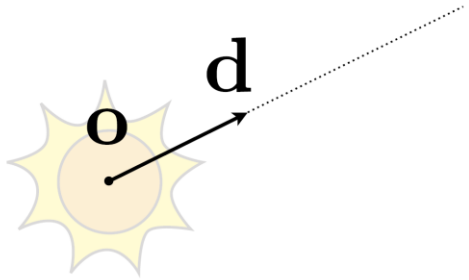
Modeling Light: Key Ideas

- Three basic ideas about light rays:

1. Light travels in a straight line
2. Light rays do not interact with each other
3. Light travels from light sources to the eye, but physics is invariant to the reversal of the direction of propagation – reciprocity)

Ray Equation

- A ray is a semi-infinite line defined by an origin $\boldsymbol{o}$ and a direction vector $\boldsymbol{d}$



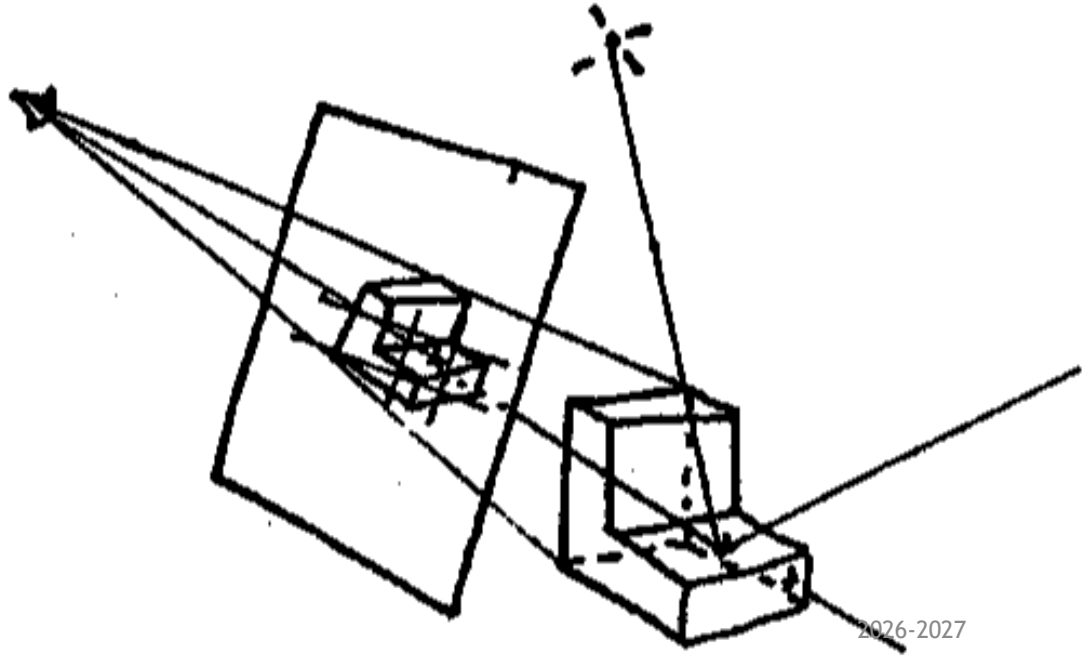
- (Parametric) ray equation:

$$\boldsymbol{r}(t) = \boldsymbol{o} + t\boldsymbol{d}, \qquad 0 \leq t < \infty$$

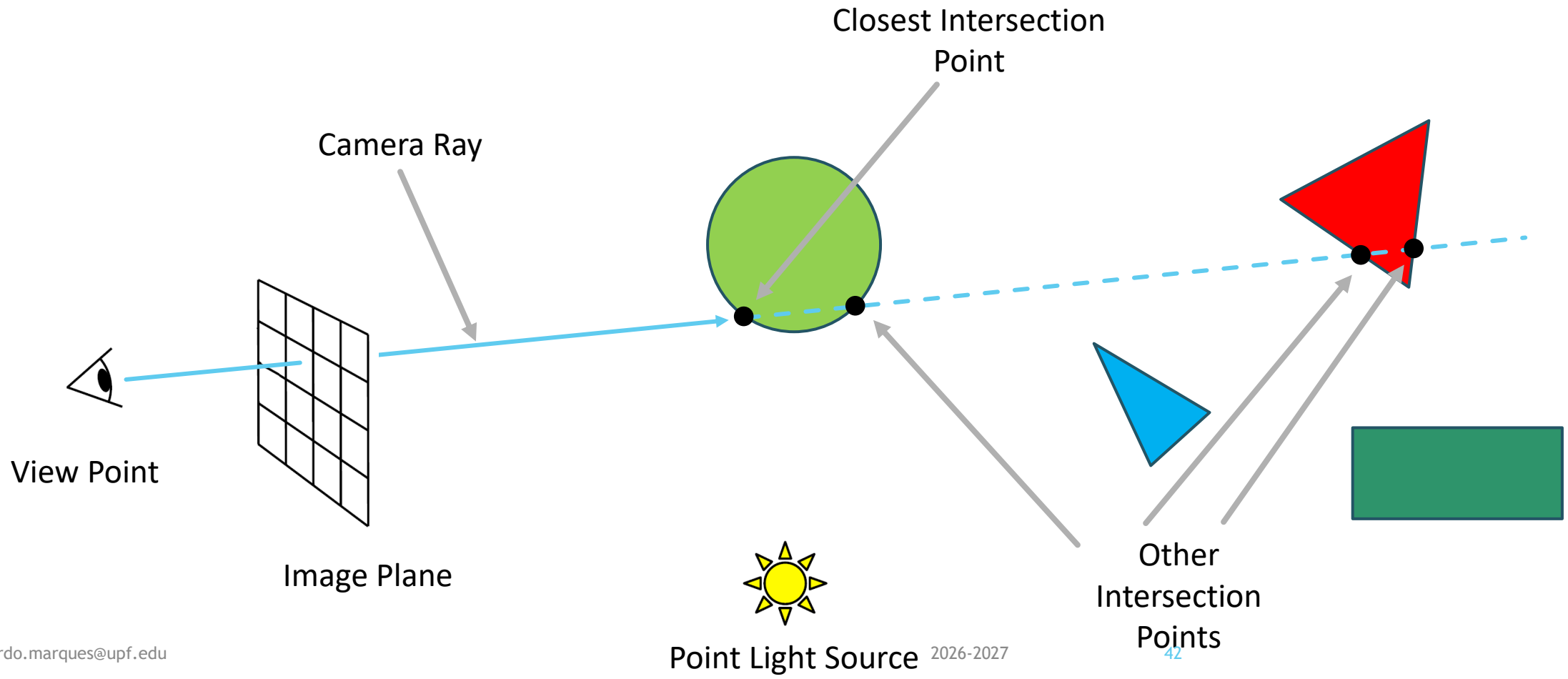
- t is the "ray parameter" (scalar) and yields the position along the ray

Ray Casting

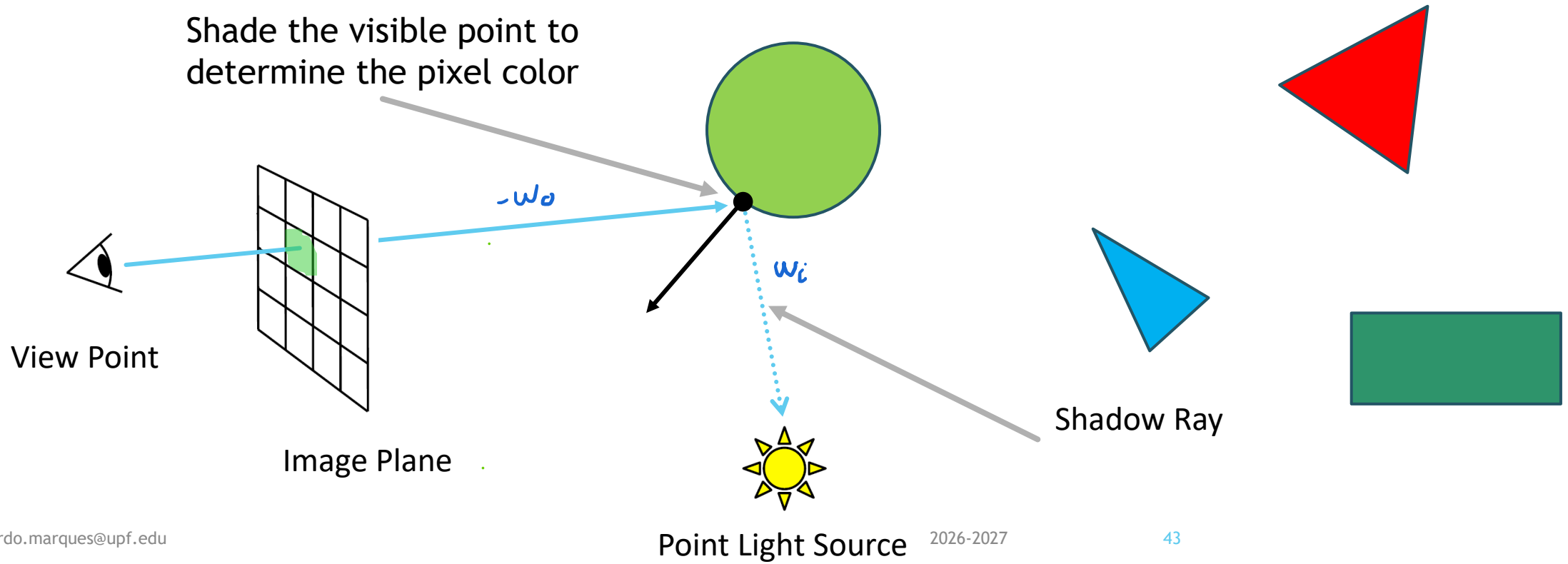
- Apple 1968 – Ray Casting
  1. Synthesize an image by sending one ray per pixel
  2. Determine shadows by sending a shadow ray the light source

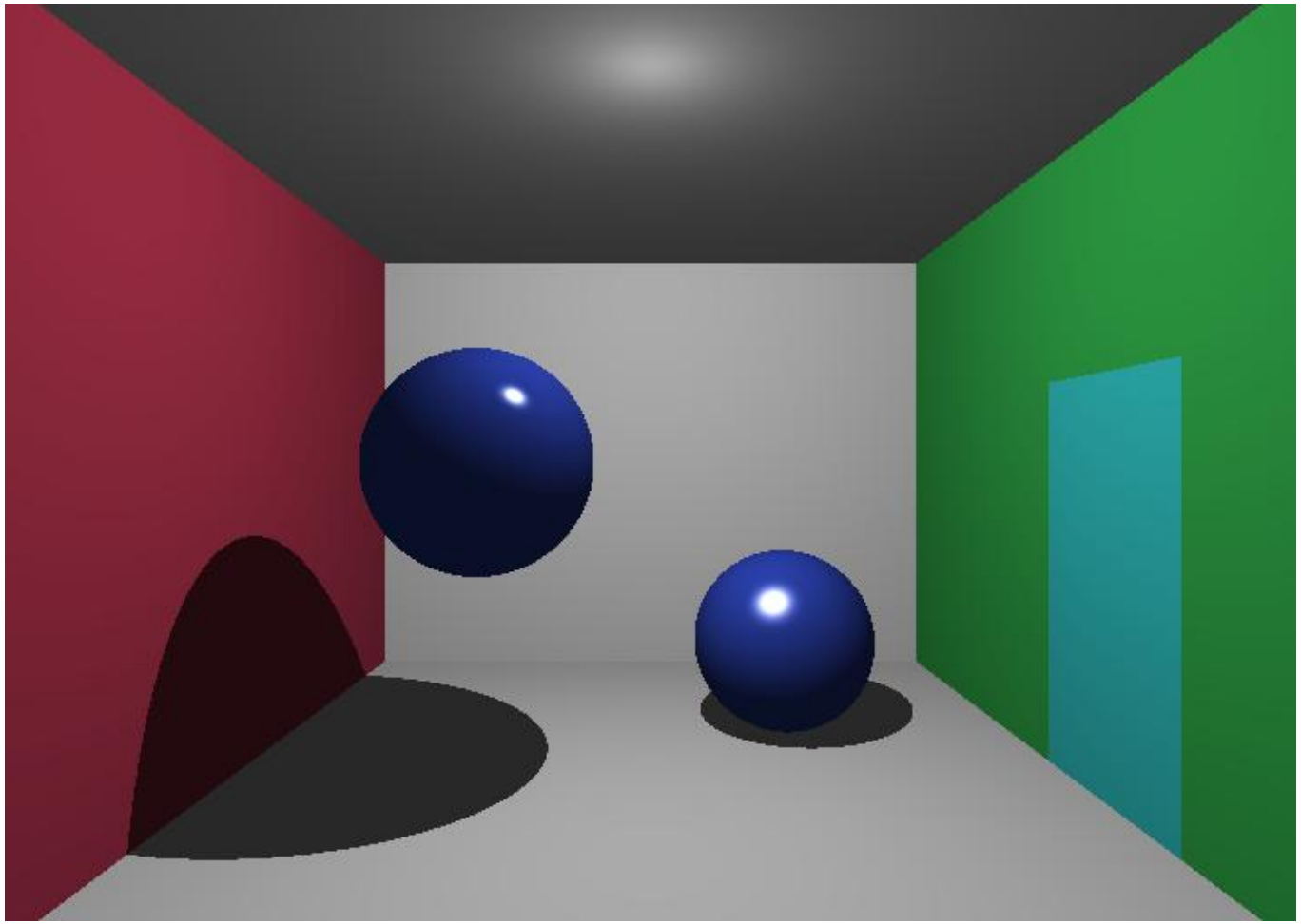


Ray Casting – Visibility



Ray Casting – Shading (Local)

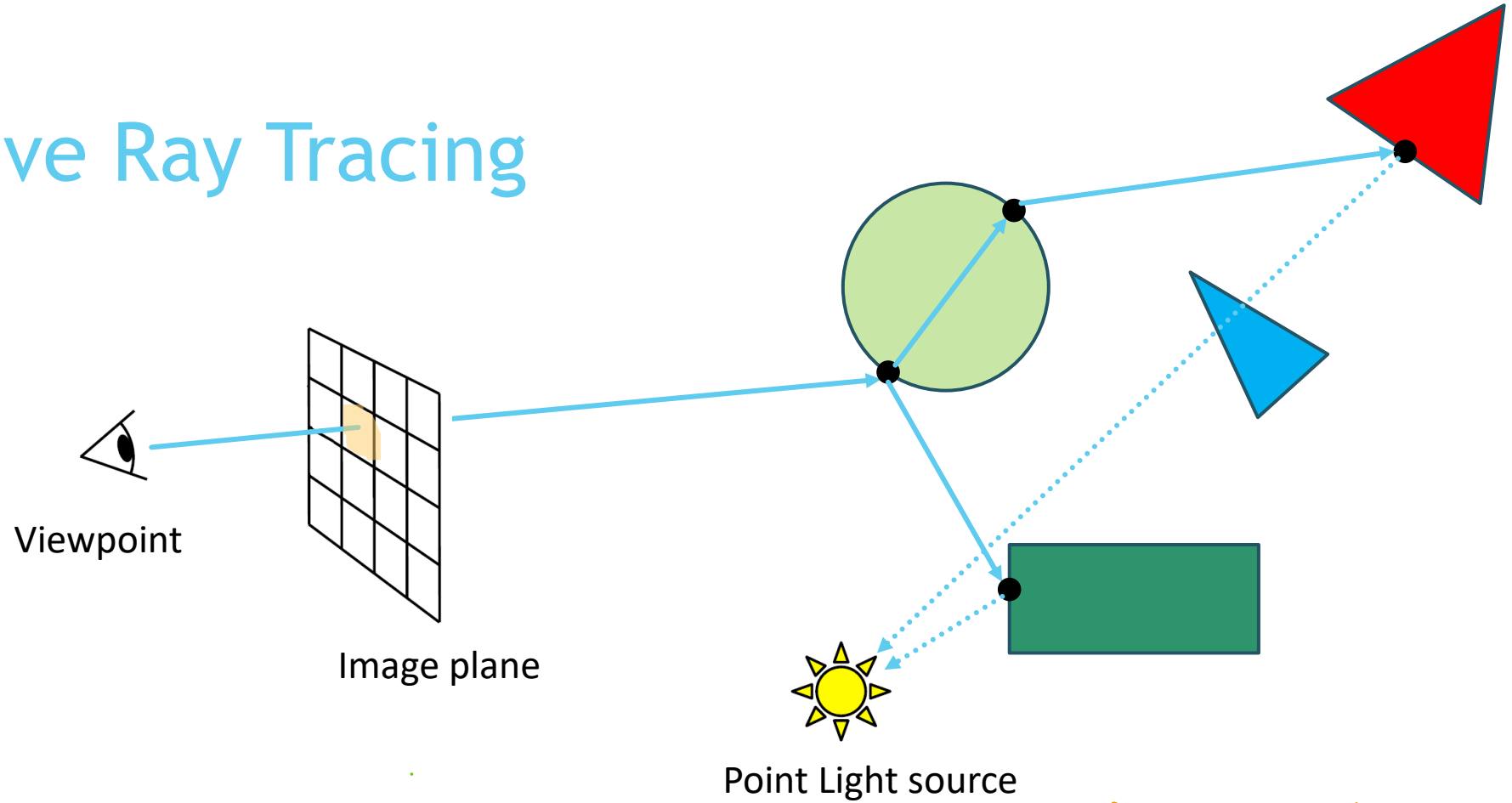




Recursive Ray Tracing

T. Whitted, 1980

Recursive Ray Tracing

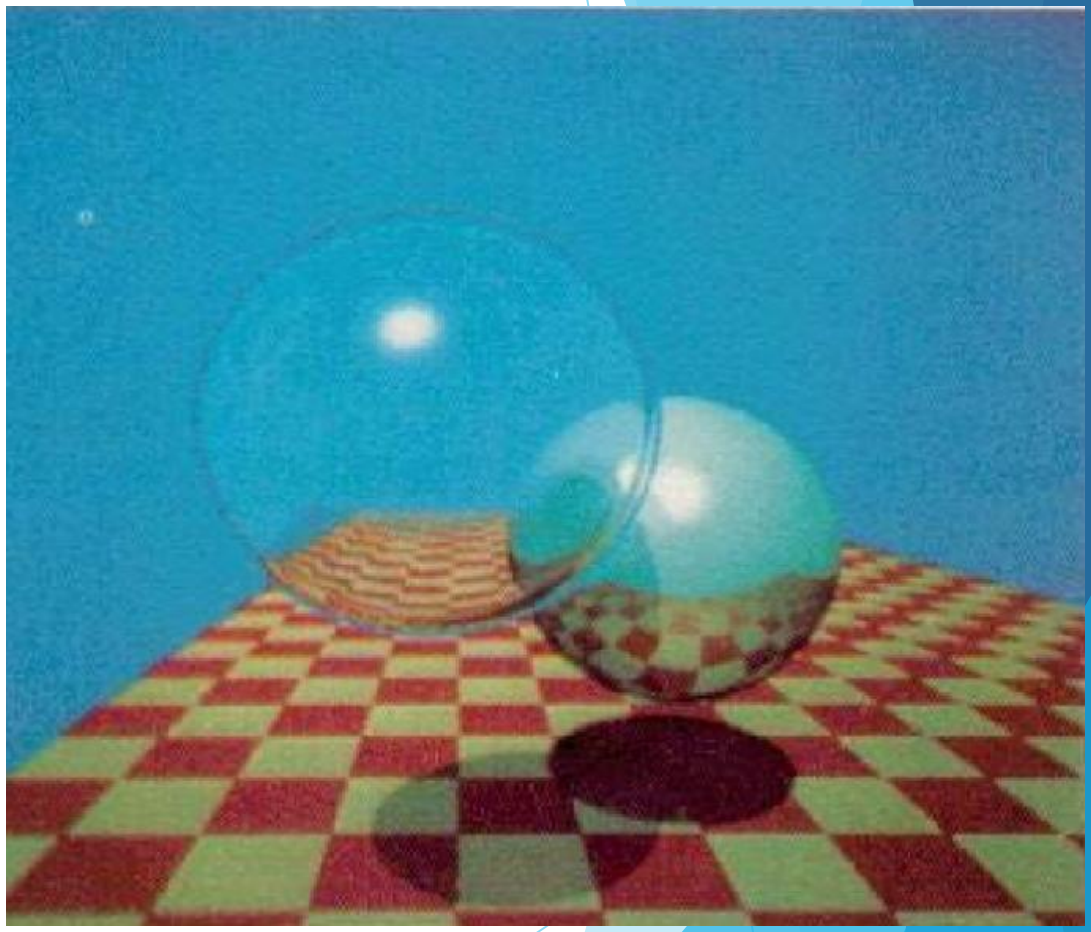


- Recursively trace secondary rays **until a non-specular surface is hit**
- At each intersection point, evaluate light sources visibility (no contribution if occluded)
- Final pixel color results from all light contributions along the paths
- Allows simulating specular reflections and transmissions

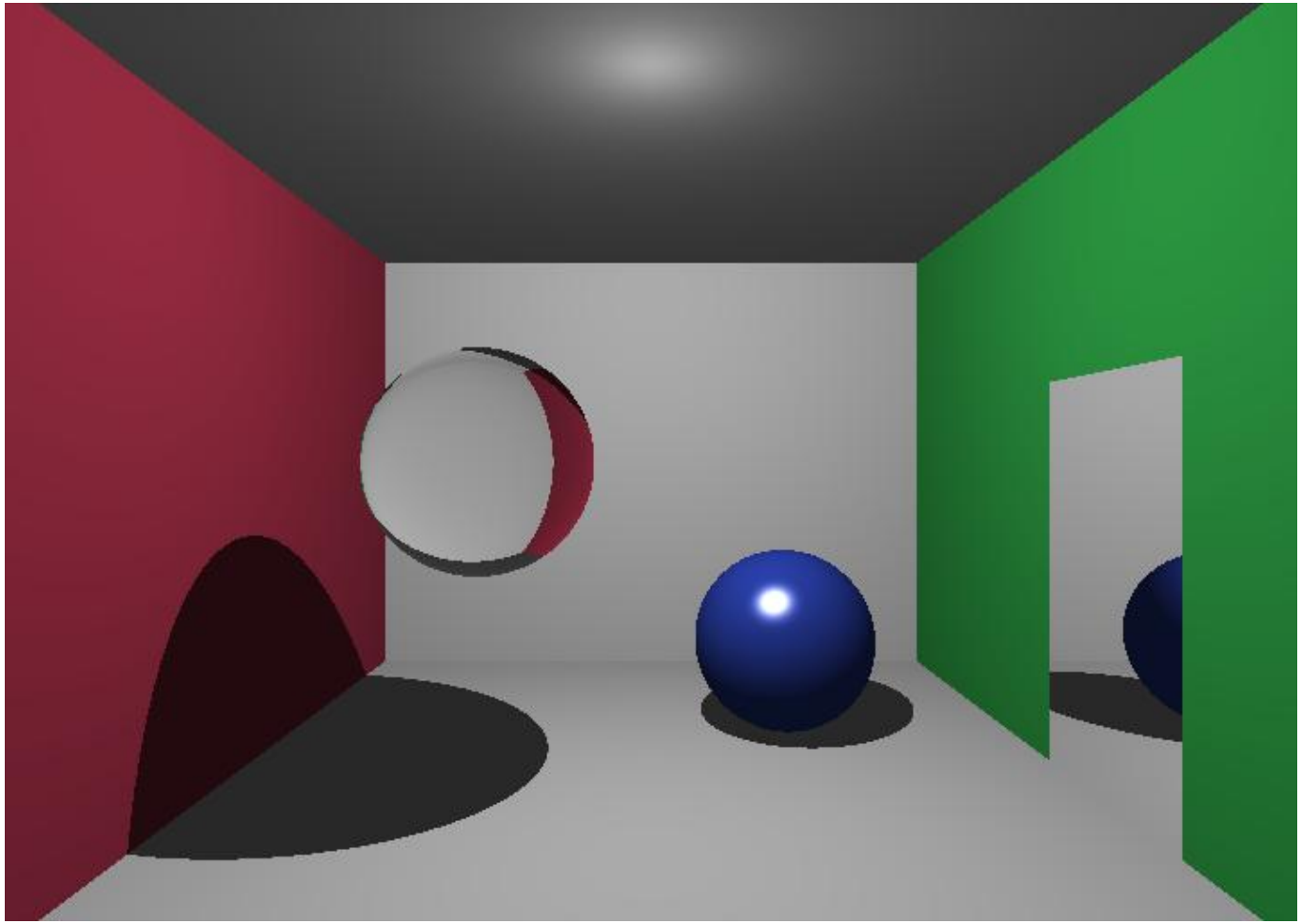
Recursive Ray Tracing

- "An improved Illumination model for shaded display", T. Whitted, CACM 1980

- Runtime:
  - VAX 11/780 (1979) 74m
  - PC (2006) 6s
  - GPU (2012) 1/30s



Spheres and Checkerboard, T. Whitted, 1980



Distribution (Stochastic) Ray Tracing

Cook, 1984

Distribution (Stochastic) Ray Tracing

- **Key idea:**
  - Shoot **several secondary rays** instead of shooting a single reflection ray
  - The contribution of the different secondary rays is averaged

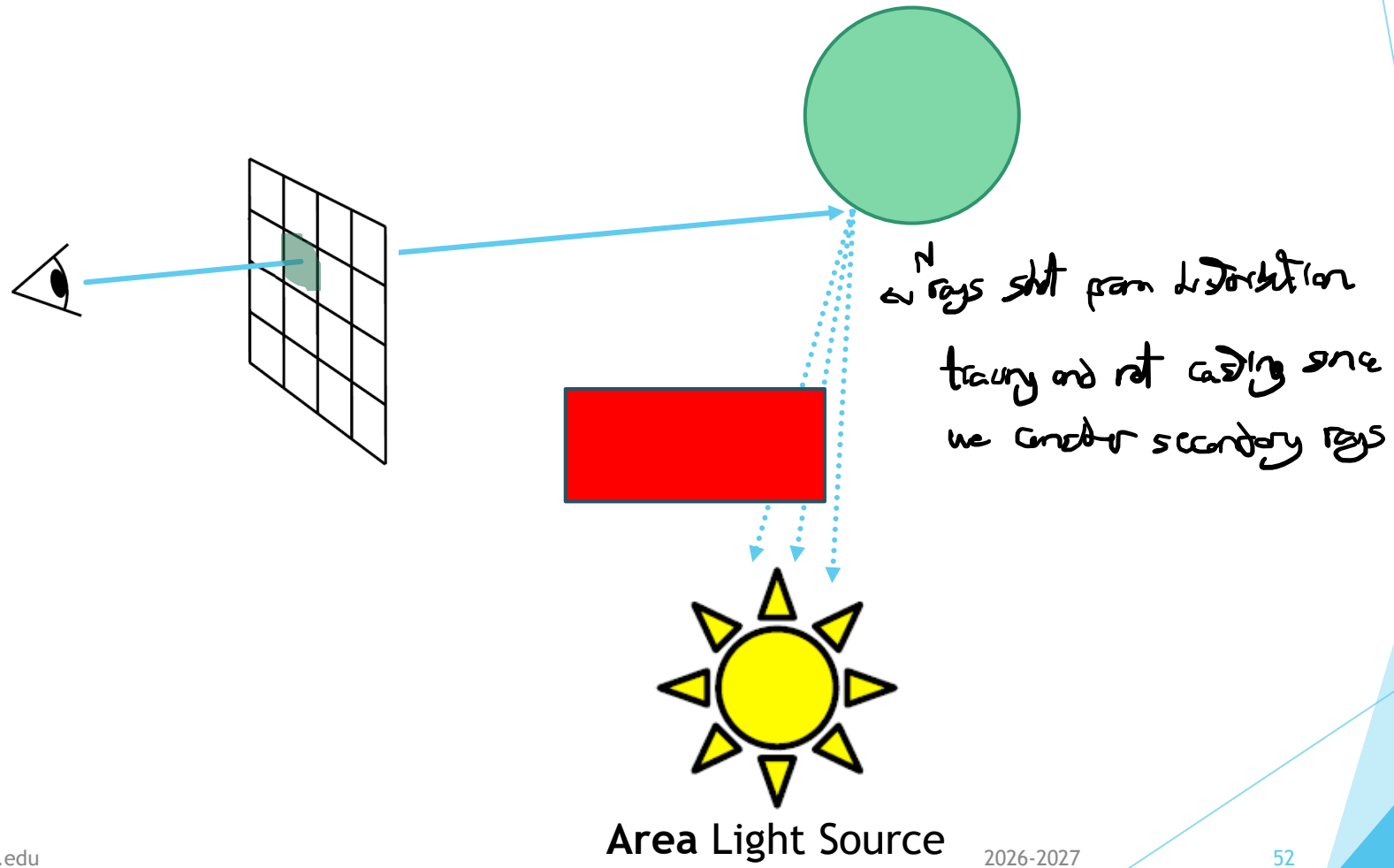
Blurred light
↳ All incident rays are still the same, but bouncy rays are all different

- Enables several interesting **light effects:**
  - **Soft shadows** (umbra and penumbra)
  - **Glossy** materials (somewhere in-between diffuse and mirror)
  - **Motion blur**, **depth of field**

↳ More accurate due to area light

- **More rays** need to be shot ➔ **slower**

Distribution Ray Tracing – Soft Shadows



Distribution Ray Tracing – Soft Shadows

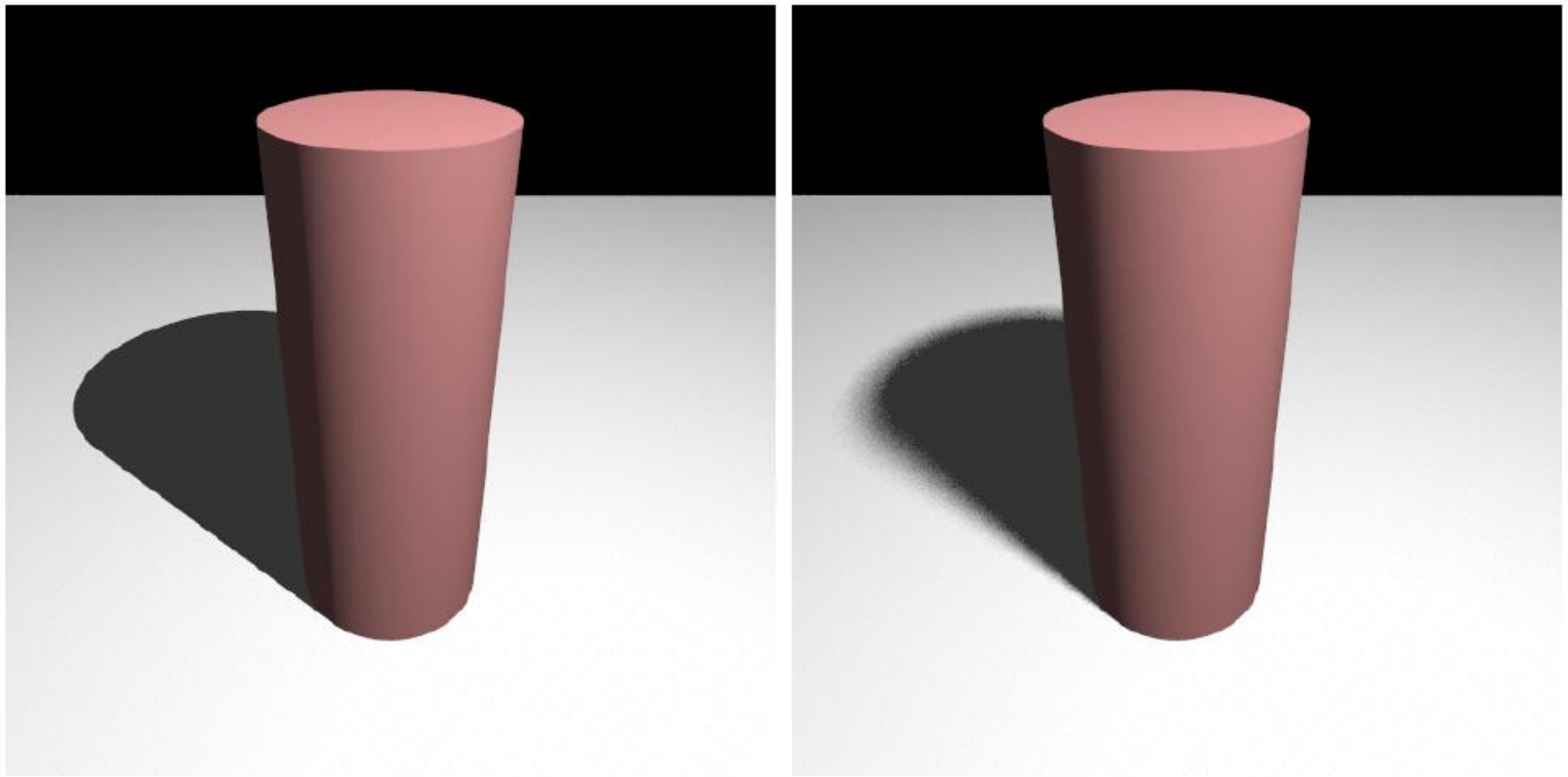
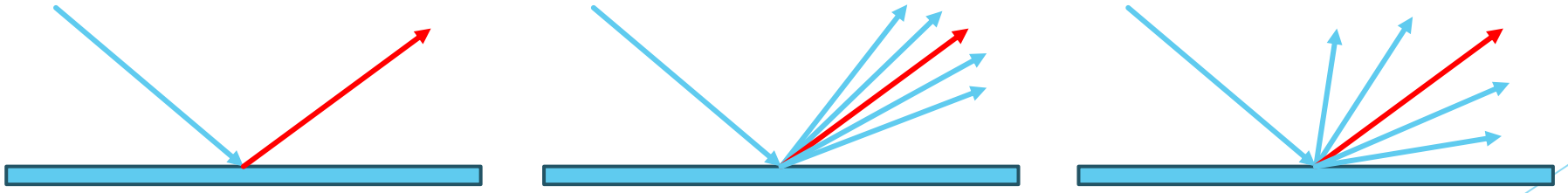


Image from Interactive Distribution Ray Tracing (Boulos et al., 2006)

Distribution Ray Tracing – Glossy Materials

V-Ray Materials





Depth of Field

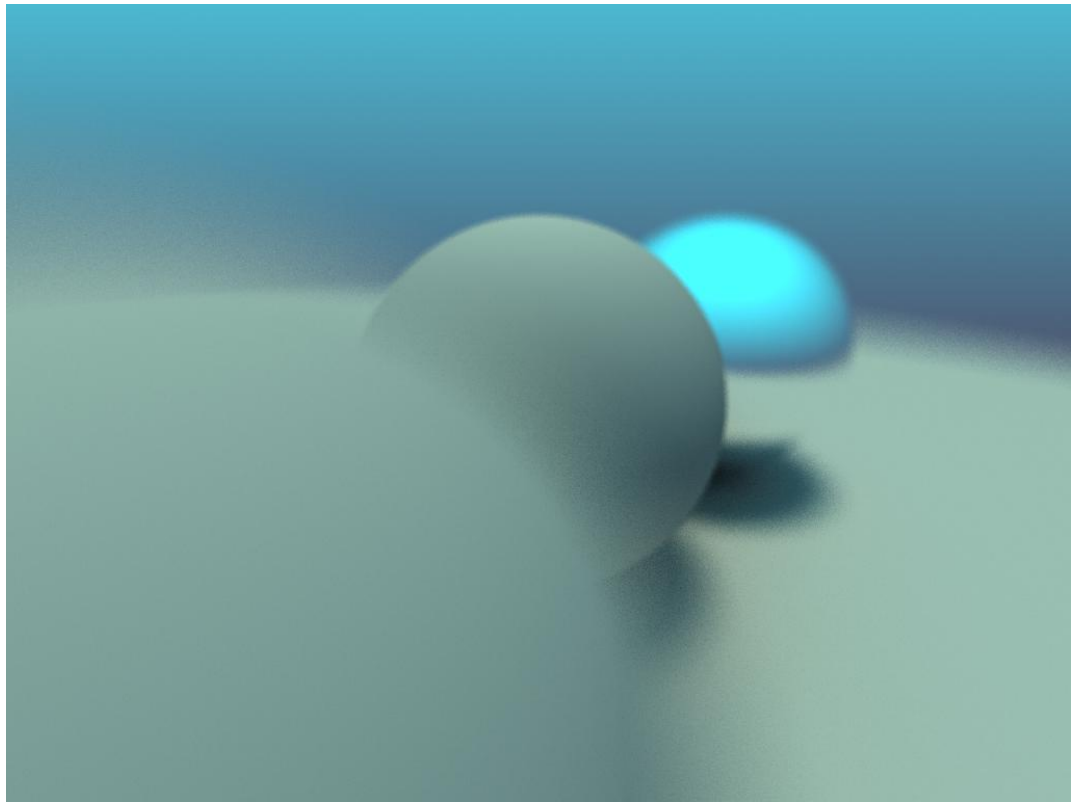


Image from Peter Shirley's Graphics Blog

- Caused by the use of a camara lens
- Only the objects within a certain distance range are properly focused
  - Objects before and after this range take a blurry aspect
- This effect can also be simulated using ray tracing
  - More details here if you're curious (Chapter 11 of P. Shirley's book "Ray Tracing in One Weekend")

Path Tracing

Kajiya, 1986

Path Tracing

- Key ideas
  - Trace **random paths** from the camera throughout the scene
  - The paths can have a varying number of segments and do not end when a diffuse material is hit
  - Several paths are generated for each pixel
  - The paths' contribution is averaged to get the final pixel color
- Generalized the previous rendering algorithms into a single framework
  - Based on the Rendering Equation (Kajiya, 1986)

Path Tracing

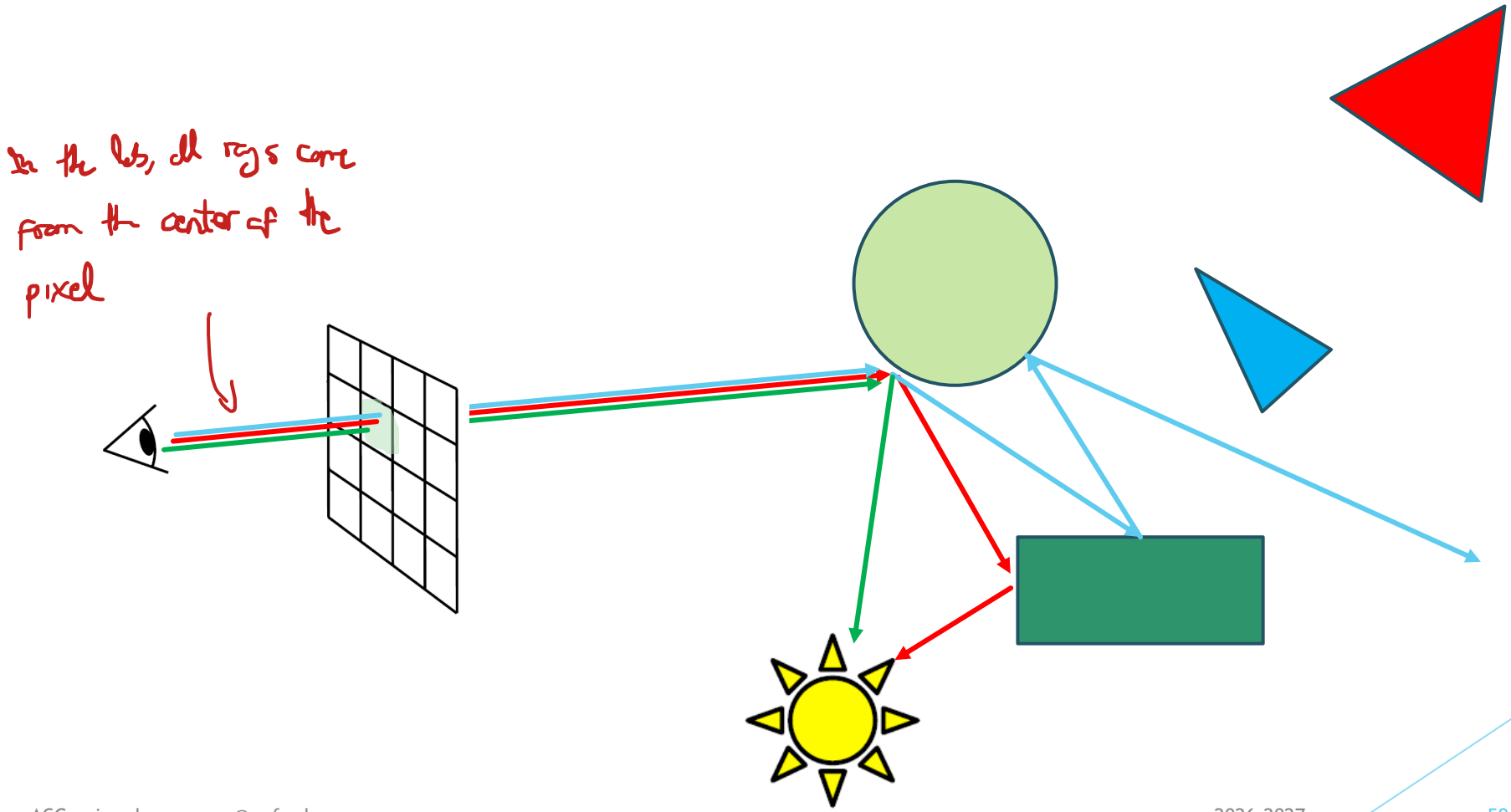




Image from ACCA Software

Summary

- We have seen several ray tracing-based algorithms with increasing complexity
  - Ray Casting (Apple, 1968)
  - Recursive Ray Tracing (T. Whitted, 1980)
  - Distribution Ray Tracing (Coock, 1984)
  - Path Tracing (Kajiya, 1986)

- Our goal
  - Acquire the theoretical and practical knowledge for *developing your own path tracer*
  - This session: ability to perform ray-surface intersection (Part II of this lecture)

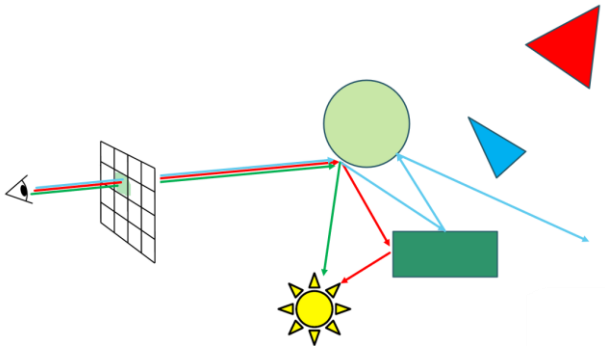
Part II

(Pause)

Ray-Surface Intersection

Introduction and Motivation

Motivation – Ray-Scene Intersection



- 3D scene objects typically represented by (triangle) meshes
  - Set of triangles that approximate a real-world shape
  - The larger the number of triangles, the more precise the shape approximation
- Ray tracing requires ray-object (thus, ray-mesh) intersection tests
  - Visibility, shaddows, illumination…
- **Idea**: simply intersect the ray with each triangle of the mesh
  - Simple, but slow…

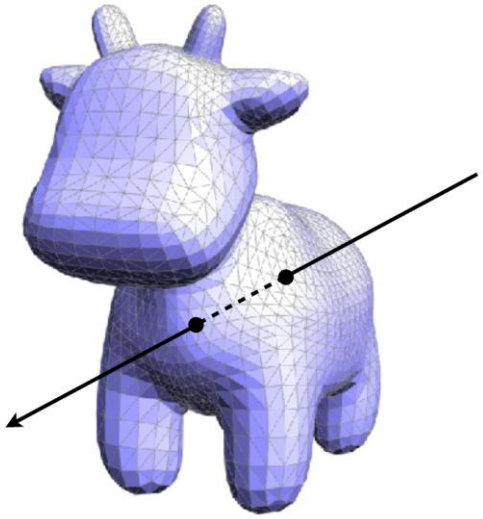


Image from Berkeley CS184/284A

Exercise 1 (to be worked on later)

Consider the mesh associated to a solid tetrahedral object whose vertices are at the coordinates $A = (4, 4, 4)$, $B = (7, 4, 4)$, $C = (4, 8, 4)$ and $D = (4, 4, 9)$. A camera is located at the position $E = (2, 2, 2.5)$.

1. Define the four triangles of the mesh with its correct orientation

2. Build the corresponding four outer normals

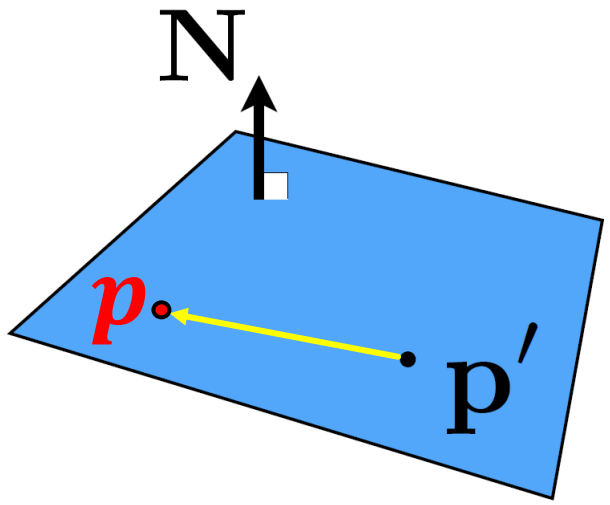
3. Consider a ray with origin E and direction $d = (0.57711, 0.69253, 0.43283)$. Without explicitly computing the inside/outside triangle test, determine the eventual intersections of the ray with the triangles of the mesh.

4. Using the inside/outside triangle test, show that the intersection point between the ray and the plane containing the triangle BAD is outside the triangle BAD

Ray-Plane Intersection

Plane Equation

- A **plane** is defined by a **normal vector (N)** and a **point in the plane (p')**
- Example:



- Plane Equation:

$$\boldsymbol{p} : (\boldsymbol{p} - \boldsymbol{p'}) \cdot \boldsymbol{N} = 0$$

Parametric equation

$$ax + by + cz = 0$$

Ray-Plane Intersection

- **Ray equation**

  $\boldsymbol{r}(t) = \boldsymbol{o} + t\boldsymbol{d}, \quad 0 \leq t < \infty$

- **Plane equation**

  $\boldsymbol{p} : (\boldsymbol{p} - \boldsymbol{p}') \cdot \boldsymbol{N} = 0$

- **Compute the ray-plane intersection**

  We write $\boldsymbol{p} = \boldsymbol{r}(t)$, and determine t

  $(\boldsymbol{p} - \boldsymbol{p}') \cdot \boldsymbol{N} = (\boldsymbol{o} + t\boldsymbol{d} - \boldsymbol{p}') \cdot \boldsymbol{N} = 0$

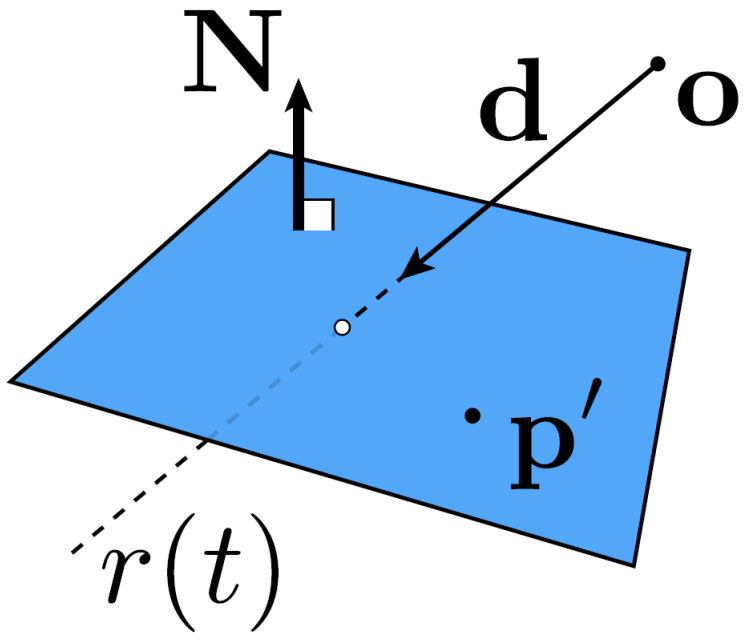
  checks if $(o+td)$ is a point in the plane

  $t = \frac{(\boldsymbol{p}' - \boldsymbol{o}) \cdot \boldsymbol{N}}{\boldsymbol{d} \cdot \boldsymbol{N}}$, verify that $0 \leq t < \infty$

$o + td - p' = N$

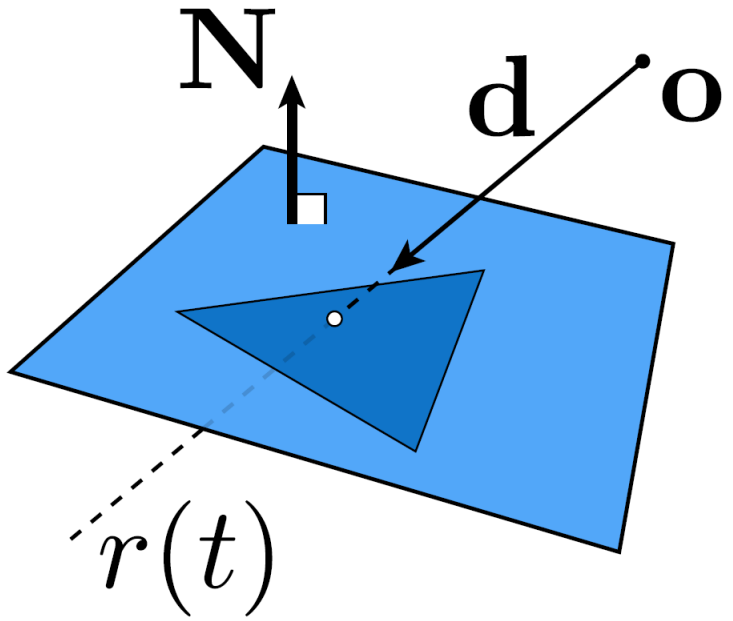
$t = \frac{p' - o}{d}$

with t solved, intersection point is



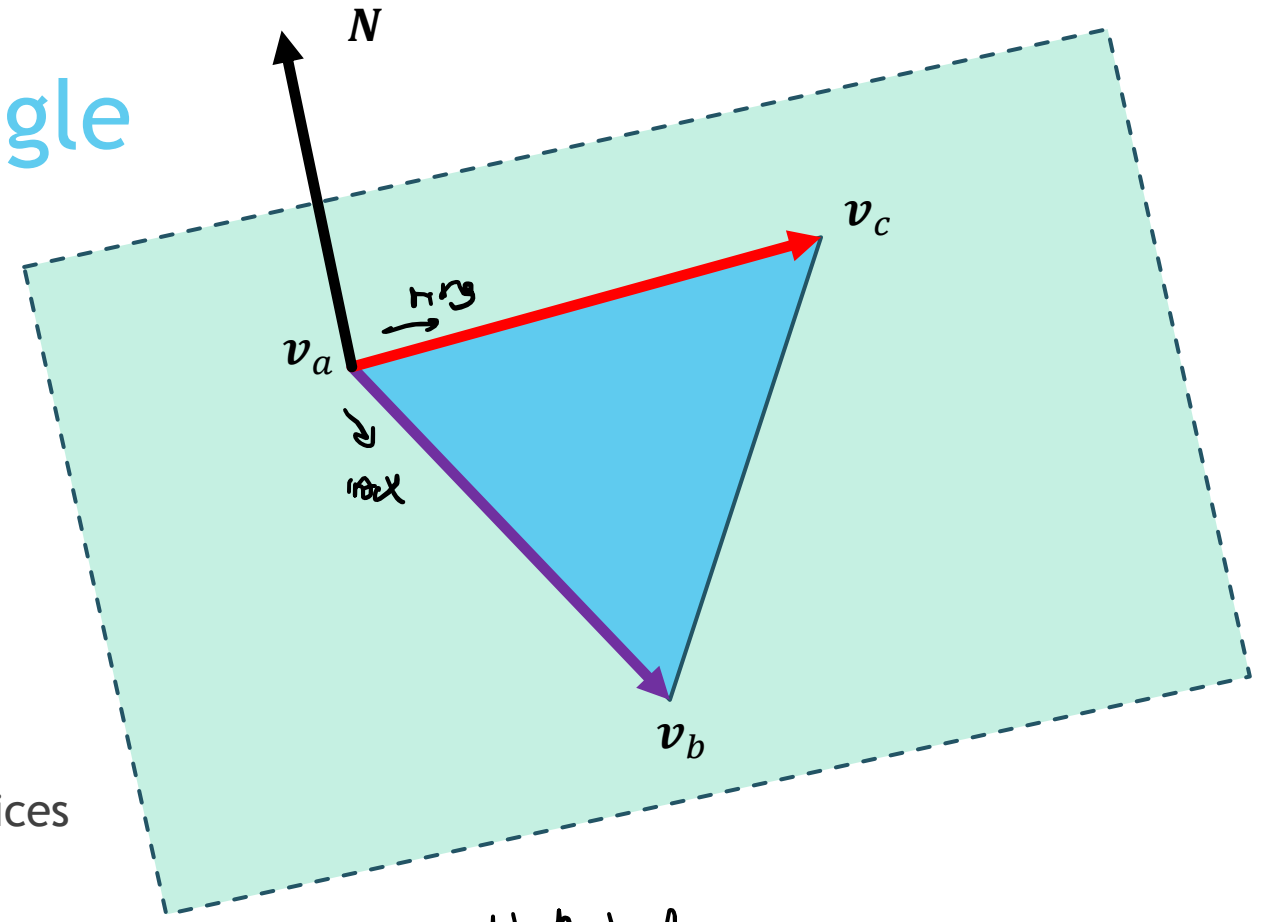
Ray-Triangle Intersection

Ray-Triangle Intersection



- Observation: a **triangle** is always contained within a plane
- Split the problem to have a two-step solution
  - **Step 1**: Compute the intersection of the ray with the plane containing the triangle
    - We already know how to do this… do we?
  - **Step 2**: Determine whether the intersection point is inside or outside the triangle

Plane Containing the Triangle



- Consider a triangle with vertices v_a, v_b and v_c

All 3 must be on the plane

- Recall the plane equation

  $p : (p - p') \cdot N = 0$

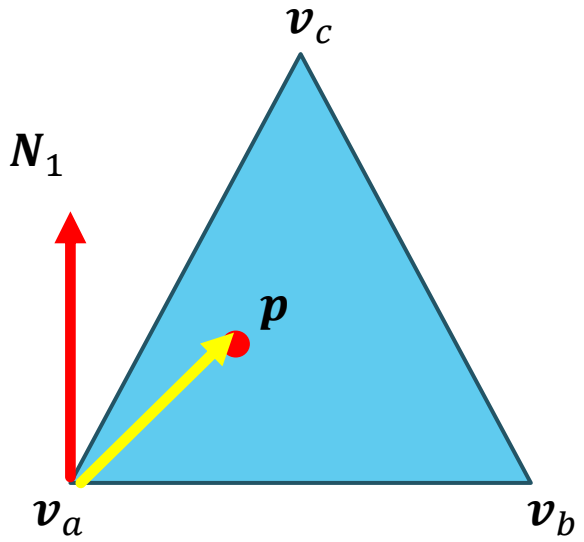
  - Need p' and N

- Point p' of the plane can be any of the triangle vertices
  - Recall the the plane contains the triangle

- What about the normal N ?
  - Can be obtained using the cross (or vectorial) product, s.t. $N = (v_b - v_a) \times (v_c - v_a)$
  - Right-hand rule: the order matters!

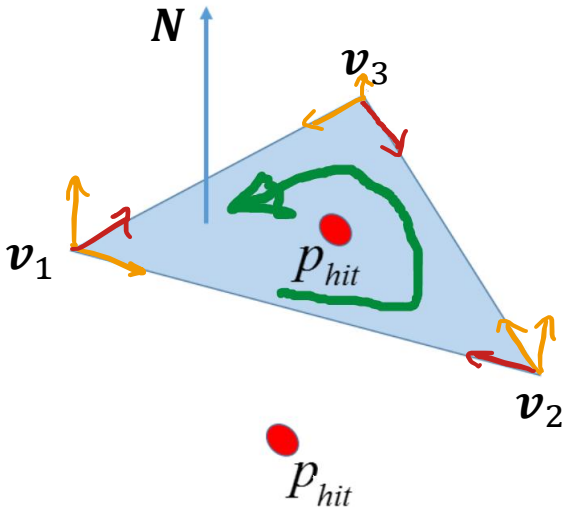
right hand rule
index → first vector
ring → second vector
thumb up (pointing outside)

Inside/Outside Triangle Test (2D)



- How to determine whether the point p ● is inside or outside the triangle?
- Recall the 2D inside/outside test from the Computer Graphics course
- Consider the following triangle and its edges
  - Step 1: Build normal (perpendicular) vector to each triangle edge, pointing inwards
  - Step 2: Use the dot product to check whether the point is on the “good” side of the edge
    - If the dot product is smaller than zero, then the point is outside the triangle

Inside/Outside Triangle Test (3D)



- Extend the 2D inside/outside test to 3D
  - Step 1: Build a normal to each side of the triangle pointing inwards using vector products

$$\boldsymbol{N}_1 = \boldsymbol{N} \times (\boldsymbol{v}_2 - \boldsymbol{v}_1) \qquad \boldsymbol{N}_2 = \boldsymbol{N} \times (\boldsymbol{v}_3 - \boldsymbol{v}_2) \qquad \boldsymbol{N}_3 = \boldsymbol{N} \times (\boldsymbol{v}_1 - \boldsymbol{v}_3)$$

  - Step 2: Use the normals to check “In Triangle”. The following scalar products should all be positive:

$$\boldsymbol{N}_1 \cdot (\boldsymbol{p}_{hit} - \boldsymbol{v}_1) \qquad \boldsymbol{N}_2 \cdot (\boldsymbol{p}_{hit} - \boldsymbol{v}_2) \qquad \boldsymbol{N}_3 \cdot (\boldsymbol{p}_{hit} - \boldsymbol{v}_3)$$

Exercise 1

Consider the mesh associated to a solid tetrahedral object whose vertices are at the coordinates $A = (4, 4, 4)$, $B = (7, 4, 4)$, $C = (4, 8, 4)$ and $D = (4, 4, 9)$. A camera is located at the position $E = (2, 2, 2.5)$.

1. Define the four triangles of the mesh with its correct orientation

2. Build the corresponding four outer normals

3. Consider a ray with origin E and direction $d = (0.57711, 0.69253, 0.43283)$. Without explicitly computing the inside/outside triangle test, determine the eventual intersections of the ray with the triangles of the mesh.

4. Using the inside/outside triangle test, show that the intersection point between the ray and the plane containing the triangle BAD is outside the triangle BAD

Ray Tracing: Performance Challenges

- Basic ray-scene intersection:
  - Test intersection with all the scene objects
- How costly is this?
  - #intersection tests $\geq$ #pixels $\left(\approx 10^6\right)$ x #triangles (?)
  - How many triangles on a complex scene?


San Miguel’s Scene, 10.7M triangles
Jun Yan, Tra


Plant Ecosystem, 20M triangles
Deussen et al; Pharr & Humphreys, PBRT

Ray Tracing: Performance Challenges

- Basic ray-scene intersection:
  - Test intersection with all the scene objects
- How costly is this?
  - #intersection tests $\geq$ #pixels $(\sim 10^6)$ x #triangles $(\sim 10^6)$
- Problem!
  - Extremely computing-intensive approach
  - Needs improvement
  - Suggestions?

Conclusion

Take-Away Messages

- Ray Tracing and Rasterization – Two distinct approaches to rendering
  - Ray-Tracing: pixel-ordered
  - Rasterization: object-ordered
- Ray Tracing can lead to more advanced visual effects at the cost of more computations
- The most frequent ray racing operation is ray-scene intersection, so if we want to decrease the computational time, we must improve this aspect
  - More details next class, but think about possible ideas

Calendar

Week no.	Date	Theory	Practice	Seminar	Deliveries
Week 1	28 Sep 02 Oct	**T1**: Introduction	**P1**: Introduction to A1.1	-	
Week 2	05 Oct 09 Oct	**T2**: Basic Ray Tracing		**S1**: Work on A1	
Week 3	12 Oct 16 Oct	**T3**: Ray Casting, Recursive Ray Tracing	**P2**: Introduction to A1.2		
Week 4	19 Oct 23 Oct	**T4**: Rendering Equation, Monte Carlo Integration and Direct Illumination **T5**: Indirect Illumination and Path Tracing		**S2**: Work on A1	**D.A1**
Week 5	26 Oct 30 Oct	**T6**: Volume Rendering: Absorption-Only Model	**P3**: Introduction to A2.1		
Week 6	02 Nov 06 Nov	**T7**: Volume Rendering: Emission, Absorption and Scattering Model **T8:** Neural Radiance Fields and Topics' Presentation		**S3**: Work on A2	
Week 7	09 Nov 13 Nov	**T9**: **Midterm**	**P4**: Introduction to A2.2		
Week 8	16 Nov 20 Nov		**P5**: Work on A2	**S4**: Work on A2	**D.A2**
Week 9	23 Nov 27 Nov	**T10**: **1st Presentation**	**P6**: Work on FP	-	
Week 10	30 Nov 04 Dec	**T11**: **2nd Presentation** **(First Session)** **T12**: **2nd Presentation** **(Second Session)**		**S5**: Work on FP	**D.FP**

Credits

- Thank you to Josep Blat for making available the slides for ACG (2023)
- The Ray-Tracing Algorithms' slides and images are inspired on the NVidia's Ray-Tracing Essentials series (Eric Haines)
- Several slides and images on ray-object intersection are from Berkeley's CS184/284A (excellent) course on Computer Graphics



The End

Questions?